



Internship Report

Name of Company

Student Name

Student card number: 00000000

Under the supervision of

Name of DEPA academic supervisor
DEPA academic supervisor

Name of Company supervisor
Company supervisor

Academic Year 2025–2026



I Executive Summary

[Summarise the whole report: state all the major points of your project, and outline the report's scope, purpose and conclusions.]





I Acknowledgements

[Optional. If used, it must come after the Executive Summary and before the Table of Contents.]



I Table of Contents

Introduction	6
Background	6
Data representation	6
Related work	8
Geometric models of the airway tree	8
Representation learning and generative models of vascular trees	10
Branch matching and anatomical labeling of airway trees	11
Morphological and topological analysis of airway trees	12
Datasets	13
The ATM'22 dataset	13
The AIIB23 dataset	14
Registration	15
Template construction	15
Input data and image geometry	16
Pairwise registration with SyN	17
Groupwise template construction with SyGN	20
Affine registration	23
Encoding	25
Variational autoencoder	25
Motivation	25
Model architecture	26
Loss function	27
Training	29
Results	29
Mitigating overfitting	29
Deterministic and sparse autoencoders	30
Deterministic autoencoder	30
Sparse encoder	31
Results	31

Implicit encoding	32
Motivation	32
Model architecture	33
Loss function	34
Data preparation and training	35
Evaluation metrics	35
Results	36
Labelling	36
Task formulation	36
Training data	37
Model architecture	37
Training	39
Inference	40
Reported performance	40
Application to our airway trees	40
Morphological Metrics	41
Overview	41
Common pre-processing	42
Branch-level descriptors	43
Subtree-level descriptors	44
Application to our airway trees	45
Clustering	45
Conclusions and Recommendations	45
Conclusions	45
Recommendations	46
Appendices	47
Appendix A: Title of the Document	47
List of References	48

I Introduction

[Briefly outline the company and/or department you worked for. Summarise the work you have done and clearly state the objectives of your project. You can also state the importance of your project and its relevance.]

I Background

This section first describes the ways a 3D airway tree can be represented (Section “Data representation”), then reviews related work (Section “Related work”).

II Data representation

A 3D pulmonary airway tree can be stored in several ways. The choice matters as it fixes what a model sees (a volume, a surface, sample points or a skeleton), how much memory it needs, and whether the branching structure is explicit or has to be inferred. Figure 1 shows the same tree in six common representations, and Table 1 summarizes their main properties.

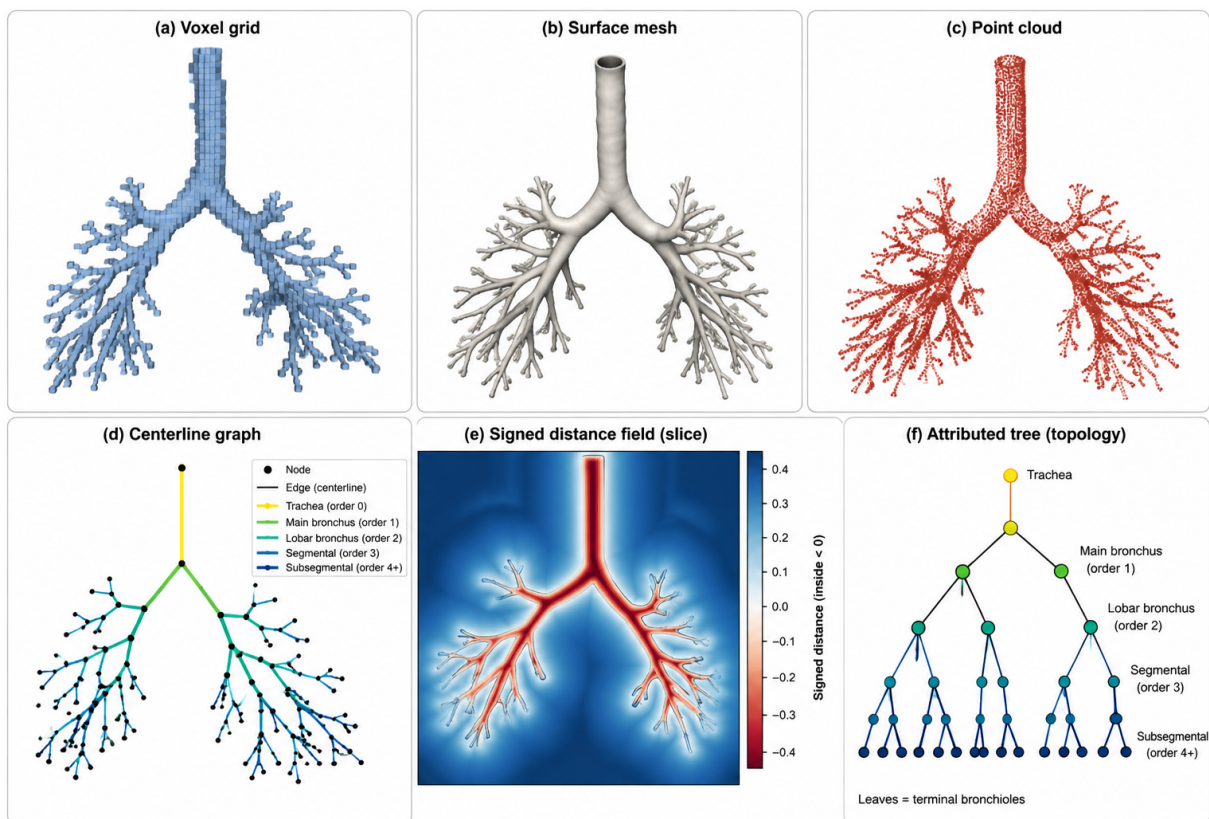


Figure 1: The same synthetic airway tree (trachea and three generations) in six representations: (a) binary voxel grid, (b) triangulated surface mesh, (c) point cloud sampled on the surface, (d) centerline graph, with edge colour and width following the branch radius, (e) signed distance field on a slice through the first bifurcation, with the airway surface as the zero level set (black line), and (f) the abstract tree topology, whose edges carry attributes such as radius r and length ℓ .

Voxel grid. The airway is a binary occupancy mask on a regular 3D grid, usually the grid of the CT scan itself. This is the direct output of manual annotation and of segmentation networks, and the format in which ATM'22 and AIB23 are distributed (Section “Datasets”). It fits 3D convolutional networks naturally (Brock et al., 2016; Garcia-Uceda et al., 2021), but memory grows with the cube of the resolution while almost all voxels are empty, and the shape is limited by the voxel spacing: thin distal branches cover only a few voxels and tend to look jagged or broken (Zhang et al., 2024). Sparse variants store only the occupied voxels, for example with the VDB data structure (Museth, 2013) used by XCube (Ren et al., 2024), or with sparse convolutions (Prabhakar et al., 2026). In this work, voxel grids are used for registration and by the dense and sparse autoencoders.

Surface mesh. A surface mesh describes the boundary of the airway lumen with vertices and triangles. It is usually extracted from the voxel mask with Marching Cubes (Lorensen and Cline, 1987), or reconstructed from points, e.g. by Poisson surface reconstruction in AVATREE (Nousias et al., 2020). Meshes are compact, give sub-voxel smooth surfaces and are the standard input for flow simulation (Lauria et al., 2022; Ho et al., 2024), but they carry no explicit branch structure and can contain non-manifold elements, which is why their number is used as a quality metric (Zhang et al., 2024). In this work, meshes are extracted as the first step of the implicit model's data preparation.

Point cloud. A point cloud is an unordered set of 3D points sampled on the surface of the airway tree. It is sparse and independent of any grid, and it can be processed by point networks such as PointNet++ (Qi et al., 2017) and Point Transformer (Zhao et al., 2021). It has no connectivity, however, so thin branches may receive too few points to be recognized.

Centerline graph. Thinning the binary mask gives a one-voxel-wide skeleton (Lee et al., 1994; Palágyi et al., 2006), which can be converted into a graph whose nodes are bifurcations and end points and whose edges are the branches between them (Bumgarner and Nelson, 2022). Each edge can carry attributes such as radius, length and direction. The representation is very compact and makes the branching topology explicit, which is why it underlies most work on branch labeling (Tschirren et al., 2005; Feragen et al., 2011b; Xie et al., 2025), morphometry (Kaji et al., 2024; Zhang et al., 2025) and graph generation (Prabhakar et al., 2024). Its weaknesses are its sensitivity to skeletonization artefacts, such as spurious short branches, and the loss of the lumen shape beyond the stored attributes. In this work, the centerline graph is an input of IPGN - the model that used to generate the labels - and the basis of the morphological descriptors.

Implicit function. The surface can also be defined as the zero level set of a continuous function of space, typically a signed distance field (SDF) that is negative inside the airway and positive outside. Neural implicit representations learn this function with a small network conditioned on a latent code (Park et al., 2019; Sitzmann et al., 2020), or store it in feature planes that can be queried anywhere (Weng et al., 2026). The result is continuous, resolution-free and smooth, but a mesh has to be extracted to visualize it, and the topology is not explicit; DGCI addresses this by learning correspondences through a space shared by all trees (Zhang et al., 2024). In this work, the implicit representation is the one that produced the best reconstructions.

Attributed tree and parametric models. Finally, the airway can be reduced to a combinatorial tree whose edges carry geometric attributes, such as radius, length or landmark points along the branch. Idealized airway models describe the whole tree this way (Horsfield et al., 1971; Kitaoka et al., 1999),

tree-shape spaces use it to compare and average trees (Feragen et al., 2011a,b), and generative models encode it recursively (Feldman et al., 2023; Batten et al., 2025) or as a sequence of nodes (Feldman et al., 2025). Branch geometry can in turn be described parametrically, e.g. with NURBS (Ortiz-Puerta et al., 2022) or B-spline cross-sections (Feldman et al., 2025). Such representations are compact and interpretable, but they depend on a reliable extraction of the tree, and trifurcations or loops break the assumption of a binary tree (Miyawaki et al., 2017). The morphological descriptors used in this work operate on such a tree, whose nodes carry an anatomical label, a generation number and a length.

Table 1: Main properties of the airway tree representations, and where each one is used in this work.

Representation	Geometry	Branching explicit	Used in this work
Voxel grid	Discrete volume	No	Registration, VAE, autoencoders
Sparse voxels	Occupied voxels	No	Sparse encoder
Surface mesh	Continuous surface	No	Implicit model (data preparation)
Point cloud	Sampled points	No	Labelling (IPGN), Implicit model
Centerline graph	1D skeleton	Yes	Labelling (IPGN), morphological metrics
Implicit (SDF)	Continuous field	No	Implicit encoding (DGCI)
Attributed tree	Abstract topology	Yes	Morphological metrics

II Related work

III Geometric models of the airway tree

Morphometry and idealized models. Early airway models are based on idealized morphometry rather than on individual anatomy. Casts of human lungs were used to study branching patterns and the relation between airway lengths and diameters (Horsfield, 1986), and Horsfield et al. (1971) and Yeh and Schum (1980) proposed asymmetric models of the human bronchial tree. Kamiya et al. (1974) related airway diameter to branching angles, and Wang et al. (1992) examined deterministic bifurcating distributive systems with a Monte Carlo method. Kitaoka et al. (1999) proposed a 3D model of the human airway tree in which the branching pattern is driven by a diameter–flow relationship within an organ boundary, relating branching angle to flow rate and airway length to diameter. Hegedűs et al. (2004) gave a detailed mathematical description of airway bifurcation geometry and used it to generate surface models of idealized bifurcations, and Florens et al. (2010) presented an anatomical and functional model of the tracheobronchial tree. Tawhai et al. (2011) reviewed computational models of airway and pulmonary vascular structure and function towards a “Lung Physiome”.

Algorithmic generation of airway trees. Building on the Monte Carlo approach of Wang et al. (1992), Tawhai et al. (2000) introduced a volume-filling algorithm that grows a bifurcating tree inside each lung lobe. Davoodi and Boozarjomehry (2016) generated the human bronchial tree with a Lindenmayer system, and Abbasi and Boozarjomehry (2021) extended it to a stochastic parametric L-system with parallel growth of bronchopulmonary segments. Geronzi et al. (2024) proposed a parametric 3D model of human airways for particle drug delivery and deposition.

Subject-specific models from CT. Subject-specific airway trees can be extracted from CT images. Palágyi et al. (2006) developed the skeletonization used to obtain a 1D tree from a segmented airway, and Nakamura et al. (2008) presented automated segmentation and morphometric analysis of the

airway tree from multidetector CT. Tawhai et al. (2004) combined CT-based geometry analysis with finite element models of the human and ovine bronchial tree, using their volume-filling algorithm to extend the CT-resolved tree to the terminal bronchioles, guided by the resolved airways and the lobar boundaries. Lin et al. (2009) and Tawhai et al. (2009) combined CT-based central airways with such artificial airways to build subject-specific models from the glottis to the terminal bronchioles for CFD, but their pipeline required manual intervention to construct trifurcations; Lin et al. (2013) later reviewed multiscale image-based modeling of gas flow and particle transport in the lungs. Montesantos et al. (2013) reported HRCT-based airway morphometry in healthy subjects and asthmatic patients, and Montesantos et al. (2016) presented an algorithm for an individualized 3D deterministic model of the human bronchial tree from HRCT images, which they evaluated statistically. Bordas et al. (2015) developed a pipeline for patient-based complete conducting airway models of healthy and asthmatic subjects, covering CT acquisition, lung, lobar and airway segmentation, centerline extraction, and algorithmic generation of the distal airways beyond the CT-resolved tree.

Trifurcations are a frequent source of difficulty for these models, although they are common in real airway trees. Boyden (1949) studied bronchopulmonary segments in 50 subjects and reported a trifurcation at the right upper lobe bronchus in 46% of them, and Zhao et al. (2009) examined the left-lung bronchial anatomy of 216 subjects with multi-detector CT and found trifurcations at the left upper division bronchus and the left lower lobar bronchus in 23% and 18% of subjects, respectively.

Miyawaki et al. (2017) proposed a centerline-based method that automatically constructs subject-specific three-dimensional models of the human airways from a CT-segmented airway skeleton and surface. Their approach systematically identifies and classifies trifurcations, distinguishing true trifurcations from double bifurcations and fork-type from tripod-type configurations, and uses dedicated geometric templates to reconstruct them. The method produces models of increasing realism (straight-centerline, curved-centerline, and fitted-surface), can be extended beyond CT resolution to the terminal bronchioles, and partitions the geometry branch by branch, which facilitates automatic mesh generation for computational fluid dynamics (CFD). Applied to 16 healthy and 16 severe asthmatic subjects, the fitted-surface models achieved an average surface error of about 11% of the image voxel size, and an aerosol deposition simulation showed branch-wise deposition efficiencies within 5% of those obtained with a CT-based model.

Nousias et al. (2020) presented AVATREE for generating anatomically valid patient-specific airway tree models from CT scans. Their pipeline first segments the lungs and the visible central airways and extracts the airway centerline as a graph. It then extends the tree beyond the generations visible in the image using a space-filling algorithm which splits the lung volume with PCA based planes. From the extended centerline they reconstruct a 3D airway surface suitable for CFD simulations using Poisson surface reconstruction. The framework also produces spatial probability maps of airway generations overlaid on the CT images. It can simulate broncho-constriction as seen in asthma and COPD, through Laplacian mesh contraction. The authors validated the generated trees against published morphometric data comparing Strahler and Horsfield orders, branch lengths, diameters and branching angles.

Meshing and hybrid models. Marchandise et al. (2013) proposed an automatic method to build and mesh tubular geometries, including trifurcations, for cardiovascular and lung applications. However, it takes a surface as input and is not centerline based, so the mesh patches are not linked to branches with anatomical data. Lauria et al. (2022) developed an automatic method to generate CFD compliant triangulated meshes from segmented airways, Ortiz-Puerta et al. (2022) introduced Snakes Isogeometric Analysis based on NURBS for healthy and COPD airway trees and Ho et al. (2024) smoothed airway surface meshes with graph convolutional neural networks. Following the same idea as Bordas et al. (2015), Agujetas et al. (2020) and Ilegbusi et al. (2023) built hybrid models that couple image derived

upper airways with idealized lower airways for CFD.

Pennati et al. (2024) presented a comprehensive review of geometrical models of the human intrathoracic airways tracing their evolution from the early idealized mathematical models of the 1960s such as the symmetric Weibel model and the asymmetric Horsfield model, to modern image-based, patient-specific reconstructions derived from CT and to a lesser extent. The authors surveyed airway segmentation strategies ranging from traditional region-growing and knowledge-based methods to machine learning and deep learning approaches such as 3D U-Net architectures and outlined the pipeline required to obtain CFD compliant meshes including centerline extraction, surface smoothing and mesh generation. They also discussed hybrid models that combine CT-resolved central airways with algorithmically generated distal trees and reviewed applications in inter-subject variability studies, diagnosis, surgical planning, drug delivery and environmental exposure. Finally, they identified key open challenges including missing small airways in segmentation, limited validation on pathological scans and the need for robust, automated mesh generation suitable for clinical use.

III Representation learning and generative models of vascular trees

Data-driven approaches have recently emerged as an alternative to rule-based (fractal and space-filling) methods for synthesizing 3D vascular geometry. Feldman et al. (2023) proposed VesselVAE, a recursive variational neural network (RvNN) that encodes vessels as binary trees where each node stores a centerline point and its radius. The encoder recursively aggregates the tree into a low-dimensional latent space while the decoder reconstructs node attributes and predicts through a node classifier, whether each node branches. New vessels are obtained by sampling the latent space and converting the generated centerlines into surface meshes. Trained on the IntrA dataset the method produced samples whose radius, length and tortuosity distributions closely match real data. However, it assumes circular cross-sections and has difficulty scaling to deep trees. To overcome these limitations, the same group introduced VesselGPT (Feldman et al., 2025), which treats vascular trees as sequences of nodes obtained through a preorder traversal. A VQ-VAE first learns a discrete vocabulary of node embeddings and a GPT-2 model is then trained to generate vessels autoregressively over these tokens. Instead of a single radius each cross-section is represented with a B-spline which preserves non-circular shapes such as aneurysms. The final mesh is extracted from a signed distance field using marching cubes. Evaluated on the Aneurisk dataset, VesselGPT generates deeper and more detailed vascular trees than previous recursive and diffusion-based approaches. Graph-based generation has also been explored: Prabhakar et al. (2024, 2025) proposed two-stage discrete diffusion models that generate simplified metric graphs of vessels and airways.

Learning compact representations of vascular structures has recently gained attention with approaches differing mainly in how the vessel geometry is represented. Batten et al. (2025) proposed VeTTA, a two-stage Transformer-based autoencoder that encodes vessel trees into a single vector representation. In the first stage a Vessel Autoencoder embeds individual vessel segments described by centerline points and radii lifted to sinusoidal Fourier features into compact latent codes. In the second stage a Vessel Tree Autoencoder encodes the full set of tree edges into a global latent vector which is decoded recursively with a slot-based Transformer decoder that guarantees a topologically valid tree. Evaluated on a synthetic 2D tree dataset and a 3D coronary artery dataset, VeTTA outperformed 3D convolutional autoencoders in reconstruction accuracy required less GPU memory and produced smooth, topologically consistent latent interpolations. However, it is restricted to tree-structured data with at most two children per node.

In contrast, Prabhakar et al. (2026) proposed VAEsselSparse a variational autoencoder that operates directly on high-resolution organ-level binary vessel segmentation maps without assuming a tree struc-

ture. By combining sparse 3D convolutions with sparse windowed self-attention their model exploits the inherent sparsity of vascular masks and achieves an $8 \times 8 \times 8$ spatial compression while preserving vessel connectivity. VAEsselSparse outperformed a dense VAE and MedVAE on topology-aware metrics such as cIDice and Betti number errors. The authors further showed that the learned latent space supports downstream tasks including aneurysm/stenosis classification, circle of Willis subtype classification and the generation of realistic vessel structures with a flow-matching model trained in the latent space.

III Branch matching and anatomical labeling of airway trees

Registration, branch matching and branch labeling of airway trees have been studied for over a decade. Several successful methods are based on association graphs where branch matchings are induced by maximal cliques of a graph combining information from two trees (Kitaoka et al., 2002; Tschirren et al., 2005; Metzen et al., 2009); Tschirren et al. (2005) reported a labeling success rate of 97%. van Ginneken et al. (2008) labeled airway branches recursively starting at the trachea as part of the segmentation process assigning labels from radius, orientation and bifurcation angle (90% success rate); this approach is vulnerable to differences in tree topology. Kaftan et al. (2006) matched tree paths rather than branches which avoids topological differences but discards the information stored in the tree structure and does not yield branch labels. Bülow et al. (2006) matched airway branches using only branch shape without connectivity information (69% or 40% success rate depending on the features used).

In contrast to these methods which focus on either topology or branch geometry Feragen et al. (2011b) proposed a geodesic tree-shape model for anatomical labeling of airway branches that treats tree topology and branch geometry jointly. Each airway centerline tree is represented as a combinatorial tree whose edges carry landmark points sampled along the branches. Trees of different sizes and topologies are embedded in a common Euclidean space and different representations of the same shape are identified giving a quotient space equipped with the Quotient Euclidean Distance (QED). In this space any two trees are connected by a shortest deformation (geodesic) that can change the tree topology along the way. Since computing exact geodesics is NP-hard, the authors approximate them by limiting the number of topological changes in each lobar subtree. Branch correspondences obtained from the geodesics between an unlabeled tree and a set of expert-labeled trees are combined through majority voting to assign anatomical labels. Feragen et al. (2012) later turned these geodesic distances into a hierarchical labeling scheme: a new airway tree is labeled from the root toward the periphery by matching it against a set of manually labeled training trees.

Weng et al. (2026) proposed TopoField, a topology-aware implicit modeling framework for repairing incomplete pulmonary trees (airways, arteries, and veins) extracted from CT scans. The pulmonary tree is represented by sparse surface and skeleton point clouds which are fused through a surface-to-skeleton cross-attention module and projected onto a triplane implicit field that can be queried at arbitrary spatial locations. To avoid the need for annotated breakpoints the authors introduce TopoBreak a strategy that synthetically adds disconnections to already-incomplete trees and trains the network to recover them providing weak supervision for topology repair. On top of the shared implicit field, task-specific decoders jointly perform tree repair, anatomical labeling and lung segment reconstruction in a single forward pass.

III Morphological and topological analysis of airway trees

Local airway geometry. Early quantitative work focused on local airway geometry. Nakano et al. (2000) related the wall area percent of a segmental bronchus and the extent of emphysema to lung function in smokers, and Hasegawa et al. (2006) extended such measurements across airway generations using multiplanar reconstruction. Patel et al. (2008) used Pi10, a standardized measure of airway wall thickness, to show that wall thickening and emphysema aggregate independently in families. Oguma et al. (2015) quantified the irregularity of the lumen radius along a single airway path and found it higher in COPD than in healthy subjects and asthma.

Global metrics of the tree. Several global metrics have been derived from the segmented tree. Kirby et al. (2018) introduced the total airway count (TAC) and showed that fewer visible branches predict COPD progression. Bodduluri et al. (2018) characterized the complexity of the tree by its fractal dimension and linked lower values to morbidity and mortality in COPD, and later proposed the airway surface-area-to-volume ratio as a marker of airway remodeling associated with airflow limitation (Bodduluri et al., 2021). Smith et al. (2020) defined the airway-to-lung size ratio (dysanapsis) and showed that a smaller ratio is associated with COPD risk, while Maetani et al. (2023) studied the physiological effects of this ratio, fractal dimension, and branch count in healthy never smokers.

Statistics on tree shapes. Feragen et al. (2011a) modeled anatomical trees as points in a space of tree-like shapes that captures both branch geometry and topological structure, and studied how to compute mean trees in this space, with airway trees as the application. Sørensen et al. (2011) represented airway trees as sets of attributed branches, matched the branches across subjects and classified COPD using a dissimilarity-based embedding with a k -NN classifier. Feragen et al. (2013) proposed a family of geometric tree kernels for comparing anatomical trees whose branches carry vector-valued attributes such as branch shape, radius or airway wall area percentage. Unlike classical graph and tree kernels which rely on discrete node labels their path-based (all-pairs and root-path) and pointcloud kernels incorporate continuous geometric information while remaining efficient enough to scale to datasets of nearly 10,000 trees. Applying these kernels to airway trees segmented from low-dose CT scans of 1966 subjects they used kernel-based two-sample tests to show that airway trees of COPD patients and healthy individuals follow significantly different distributions. They also showed using SVM classification that geometry-aware kernels outperform average airway wall area percentage measures and state of the art graph kernels such as the shortest path and Weisfeiler - Lehman kernels in detecting COPD.

Topological descriptors. Belchi et al. (2018) applied persistent homology to the airway tree using height as the filtration and showed that the resulting “upwards complexity” correlates with COPD severity. Kaji et al. (2024) proposed two morphological metrics based on persistent homology to quantify tubular structures in chest CT images and applied them to the airway trees of patients with chronic obstructive pulmonary disease (COPD). They computed 0-dimensional persistent homology under two distance based filtrations. The first, the geodesic distance from the carina yields $treeH_0$, which describes the complexity of the branching structure. The second the distance from the lumen centerline yields $radialH_0$, which captures irregularities of the luminal surface in the central (1st to 5th generation) airways. On three cohorts acquired with different scanners and reconstruction kernels, both metrics were higher in COPD patients than in non-COPD smokers. In addition, $radialH_0$ predicted longitudinal FEV₁ decline independently of conventional CT measures such as emphysema extent (LAA%), airway-to-lung size ratio, and total airway count.

I Datasets

[Optional short introduction: why these two datasets were used together, e.g. ATM'22 for the variety of sites and scanners, AIB23 for fibrotic lungs.]

II The ATM'22 dataset

The Multi-site, Multi-domain Airway Tree Modeling dataset (ATM'22) was released as part of an official challenge held in conjunction with MICCAI 2022 (Zhang et al., 2023). It targets the binary segmentation of the pulmonary airway tree from chest CT, and at the time of its release it was the largest publicly available airway dataset with complete voxel-wise annotations. The reference benchmark that preceded it, EXACT'09, provided only 40 scans and never released its manual annotations, which had long limited the training and fair evaluation of data-driven methods.

The dataset contains 500 chest CT scans, partitioned into 300 training, 50 validation and 150 test cases. The test set is held privately by the organizers and is accessible only through the submission of a Docker container, which guarantees the fairness and reproducibility of the reported results. The scans originate from three sources, as summarized in Table 2.

Table 2: Composition of the ATM'22 dataset by split and by source site.

Split	Shanghai Chest Hospital	LIDC-IDRI	EXACT'09	Total
Training	226	54	20	300
Validation	20	30	–	50
Test	84 (58 COVID-19)	66	–	150
Total	330	150	20	500

For the 20 cases taken from EXACT'09, only the airway annotations are distributed; the corresponding CT volumes must be obtained separately from the original source.

The acquisition parameters are heterogeneous, which is the basis of the multi-site character of the dataset. The scans were acquired on Philips iCT 256, GE LightSpeed16 and Toshiba Aquilion scanners, the last of which appears only in the test set. Each slice is 512×512 pixels, and the number of slices per scan ranges from 157 to 1125, with a slice thickness between 0.45 and 1.00 mm and an in-plane resolution between 0.500 and 0.919 mm. The health status of the subjects ranges from healthy volunteers to patients with severe pulmonary disease. The multi-domain character comes from the 58 COVID-19 scans included in the test set: ground-glass opacity, consolidation and septal thickening alter the appearance of the lung parenchyma and blur the airway lumen, so these cases form a noisy domain on which the generalization ability of a model trained on clean scans can be assessed.

The annotations were produced in two stages. A preliminary segmentation was first obtained by ensembling, through majority voting, several airway segmentation models trained on the BAS dataset. These preliminary masks were then reviewed by three radiologists with more than five years of experience: two of them independently refined and cross-checked each scan, and the most senior radiologist validated the final annotation. The manual refinement required 60 to 90 minutes per scan and focused mainly on repairing breakages in the peripheral bronchi and removing small leakages into the surrounding parenchyma, using the original CT intensities and anatomical prior knowledge. The whole collection and annotation process took the organizers approximately one year.

The challenge evaluates submissions along two complementary axes. Topological completeness is

measured by the tree length detected rate (TD) and the branch detected rate (BD),

$$TD = \frac{T_{det}}{T_{ref}}, \quad BD = \frac{B_{det}}{B_{ref}}, \quad (1)$$

where T_{det} and B_{det} denote respectively the total length and the number of branches correctly detected in the prediction, and T_{ref} and B_{ref} the corresponding quantities in the ground truth. Both metrics are computed on the largest connected component of the prediction, a branch being counted as detected when more than 80% of its centerline voxels lie within the ground truth. Topological correctness is measured by the Dice similarity coefficient (DSC) and the precision, with sensitivity and specificity reported as complementary indicators. The official ranking is based on the unweighted mean of TD, BD, DSC and precision. This dual evaluation reflects the fact that overlap-based measures alone are insufficient for tubular structures: a prediction may achieve a high Dice score while missing a substantial number of distal branches, since the peripheral bronchi account for only a small fraction of the total airway volume.

II The AIIB23 dataset

The AIIB23 dataset (Airway-Informed Quantitative CT Imaging Biomarker for Fibrotic Lung Disease 2023) was released as part of an official challenge held in conjunction with MICCAI 2023 and hosted on the CodaLab platform (Nan et al., 2024). It is the first publicly available dataset dedicated to airway segmentation in patients with fibrotic lung disease. Earlier public airway datasets, such as EXACT'09, BAS and ATM'22, mainly contain scans from healthy subjects or from patients with common pulmonary conditions such as nodules or COVID-19.

Fibrotic lung disease is particularly challenging for automatic airway segmentation for two reasons. First, honeycombing, the presence of small cystic airspaces in the lung periphery, visually resembles distal airway branches, so models often produce false-positive predictions known as airway leakages. Second, traction bronchiectasis and the distortion of lung tissue alter airway morphology, which frequently leads to missing or discontinuous branches. AIIB23 was therefore designed to benchmark segmentation methods on these difficult cases.

Data sources and composition. The dataset gathers high-resolution computed tomography (HRCT) scans from three cohorts, acquired with different vendors and scanning protocols. The training and validation sets come from the Open Source Imaging Consortium (OSIC), an open-access database of patients with interstitial lung disease (ILD), idiopathic pulmonary fibrosis (IPF) and other respiratory diseases. The test set combines 90 fibrosis cases from the Australian IPF Repository (AIPFR) with 50 COVID-19 cases from the University Hospital of Parma. The COVID-19 cases make it possible to assess generalization to a different pathology. The patient populations are broadly comparable in age, with a median age of 68 years for OSIC, 70 for AIPFR and 65 for Parma. Table 3 summarizes the data splits.

Table 3: Composition of the AIIB23 dataset (Nan et al., 2024).

Split	Source	Task I (segmentation)	Task II (mortality)
Training	OSIC	120	95
Validation	OSIC	52	52
Test	AIPFR + Parma	140 ^a	90

^a90 fibrosis (AIPFR) and 50 COVID-19 (Parma) cases.

Image selection and preprocessing. All scans were converted from DICOM to NIfTI format using SimpleITK. Only scans with more than 120 slices and an in-plane resolution of at least 512×512 pixels were retained. Despite this criterion, the number of slices varies considerably between cases. The fibrosis test scans contain significantly fewer slices than the training data, which results in lower image quality in the coronal and sagittal planes. This represents a domain shift between training and test data.

Annotation protocol. The ground-truth airway masks were produced through a semi-automatic, multi-stage process. Preliminary labels were first generated by a deep learning model trained on the BAS dataset. Three junior radiologists then revised these labels, and two senior radiologists refined them by correcting breakages and mis-annotated voxels. When annotators disagreed, the most experienced radiologist's annotation was retained. Annotation was performed in ITK-SNAP using a drawing tablet. Inter-observer agreement on airway branches was very high (Cohen's $\kappa = 0.937$).

Tasks and evaluation. The challenge defines two tasks. Task I is binary airway segmentation of the 3D HRCT volume. It is evaluated with overlap metrics (intersection over union, IoU, and precision) and with topological metrics that are particularly relevant for thin tubular structures:

- the detected length ratio (DLR) is the fraction of the ground-truth branch length that the prediction recovers;
- the detected branch ratio (DBR) is the fraction of ground-truth branches that are correctly detected, a branch counting as detected when its IoU with the corresponding ground-truth branch is at least 0.8;
- the airway leakage ratio (ALR) quantifies false-positive predictions.

The challenge's overall accuracy score is the mean of IoU, precision, DLR and DBR. For reference, the best-performing challenge submission reached an IoU of 0.905, a DLR of 0.937 and a DBR of 0.905 on the test set.

Task II is a binary classification problem: predicting whether a patient is alive or deceased 63 weeks after the first HRCT scan. The training set contains 59 survivors and 36 deceased patients, and the validation set is balanced (26/26). The test set is strongly imbalanced, with about 87 % of survivors. Performance is measured with accuracy, AUC, sensitivity, specificity and F1-score.

I Registration

[Optional short introduction: why a common spatial reference is needed, and the two steps used to obtain it – building a population template, then registering each airway tree to it.]

II Template construction

To obtain a common spatial reference for the pulmonary airway trees, a population-specific three-dimensional template was constructed from airway segmentation volumes using the Advanced Normalization Tools (ANTs) framework and its ANTsPy interface (Tustison et al., 2021). Template construction is a *groupwise registration problem*: a common template and a set of transformations mapping the individual airway trees to it are estimated together. Unlike pairwise registration to an arbitrarily selected

subject, groupwise registration does not designate one individual airway tree as the anatomical reference; the reference itself is progressively optimised to represent the population. The ANTs framework implements this as *symmetric groupwise normalization* (SyGN) (Avants et al., 2010). The registration of individual airway trees to the finished template is described separately in Section “Affine registration”.

Sources of the method description. The description combines three sources. (i) The pairwise deformable registration algorithm, Symmetric Normalization (SyN), was introduced by Avants et al. (2008) (Section “Pairwise registration with SyN”). (ii) The groupwise template-construction strategy, SyGN, was introduced by Avants et al. (2010) (Section “Groupwise template construction with SyGN”). (iii) The template initialisation, the appearance update, the parameter defaults and the order of operations are properties of the ANTsPy implementation (`ants.registration` and `ants.build_template`) (Tustison et al., 2021); where the implementation differs from the published algorithms, this is stated explicitly. Both methods were developed and validated on T1-weighted brain MRI (cortical labelling in elderly and neurodegenerative brains, and hippocampus segmentation in epilepsy, respectively); neither was evaluated on airway segmentations.

III Input data and image geometry

Selection of the input volumes. Eight airway segmentation volumes were used: four drawn at random from ATM’22 and four from AIIB23 (Section “Datasets”). The draw used Python’s pseudo-random generator without a fixed seed, so the selected subset is not reproducible from the code alone; the file names were recorded in the run log and are listed in Table 4. The volumes were passed to

Table 4: Airway segmentations used to build the template, in input order.

Order	Dataset	File name
1	ATM’22	[TODO: file name]
2	ATM’22	[TODO: file name]
3	ATM’22	[TODO: file name]
4	ATM’22	[TODO: file name]
5	AIIB23	[TODO: file name]
6	AIIB23	[TODO: file name]
7	AIIB23	[TODO: file name]
8	AIIB23	[TODO: file name]

`ants.build_template` in the order ATM’22 first, then AIIB23. This order matters for one implementation detail: the first volume defines the voxel grid of the template (see *Initialisation* in Section “Groupwise template construction with SyGN”).

Representation in physical space. Each airway tree is a three-dimensional NIfTI image: a discrete voxel array together with spatial metadata that positions the array in physical space. A voxel is identified by a discrete index $\mathbf{i} = (i_x, i_y, i_z)^T$, whereas its anatomical location is a continuous physical coordinate $\mathbf{x} = (x, y, z)^T$, related by

$$\mathbf{x} = \mathbf{o} + \mathbf{D} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix} \mathbf{i}, \quad (2)$$

where $\mathbf{o}$ is the image origin, s_x, s_y, s_z are the voxel spacings and $\mathbf{D}$ is the direction matrix of the voxel axes; in NIfTI terminology these are encoded by the spatial affine in the image header. ANTs is built on ITK, in which registration is performed in *physical coordinates*, and ANTsPy converts the NIfTI geometry into the ITK representation of origin, spacing and direction. Differences in origin, orientation or voxel spacing between two airway segmentations are therefore taken into account when their structures are put into correspondence. The NIfTI affine defines the *initial coordinate system of each image*; it is not itself a registration.

Warping convention. An image I is treated as a function of physical position, and a transformation is applied by composition. If T is the fixed image (the template) and I the moving image, the warped image is defined on the domain of T :

$$(I \circ \Phi)(\mathbf{x}) = I(\Phi(\mathbf{x})), \quad \mathbf{x} \in \Omega_T. \quad (3)$$

Hence Φ maps each point of the *template* domain to its corresponding point in the *subject* domain, where the subject image is resampled. This is the convention used by ITK and ANTs; the inverse map Φ^{-1} takes subject points to template points.

Pairwise registration with SyN

Within every template-building iteration k , each airway segmentation I_i is registered to the current template $T^{(k)}$, which serves as the fixed image. The registration seeks $\Phi_i^{(k)}$ such that

$$I_i \circ \Phi_i^{(k)} \approx T^{(k)}, \quad (4)$$

i.e. after warping, corresponding airway structures occupy similar physical locations in template space. Inside `ants.build_template` this is the call

```
ants.registration(
    fixed=xavg, # current template T^(k)
    moving=image_i, # airway segmentation I_i
    type_of_transform='SyN'
)
```

The ANTsPy SyN preset. Despite its name, the ANTsPy preset SyN contains three stages. The two images are first aligned by their centres of mass (a translation used as initial transform), an affine transformation $A_i^{(k)}$ is then estimated, and finally the deformable symmetric normalisation $\phi_i^{(k)}$ is estimated between the template and the affinely transformed subject $I_i \circ A_i^{(k)}$. In Avants et al. (2008), by contrast, “SyN” denotes only the deformable part, and rigid and scaling differences are assumed to have been removed beforehand. With the convention of Eq. (3), the total transformation is

$$\Phi_i^{(k)} = A_i^{(k)} \circ \phi_i^{(k)}, \quad I_i \circ \Phi_i^{(k)} = (I_i \circ A_i^{(k)}) \circ \phi_i^{(k)}, \quad (5)$$

so a template point is moved first by the deformable map and then by the affine map into subject space. The affine part accounts for global translation, rotation, scaling and shear; the deformable part accounts for spatially varying differences that no single affine map can represent. (The centre-of-mass translation is folded into the affine stage.)

Registration parameters. Apart from `type_of_transform`, no argument was specified, so all other settings were ANTsPy defaults: Mattes mutual information as the similarity metric of both the affine and the deformable stage; an affine stage with (2100, 1200, 1200, 10) iterations on four resolution levels; a deformable stage with (40, 20, 0) iterations from coarse to fine; an optimiser step `grad_step = 0.2`; and Gaussian smoothing of the update field (`flow_sigma = 3`) and of the total field (`total_sigma = 0`). Because the last entry of the deformable iterations is zero, no deformable optimisation is performed at full image resolution, which limits how precisely thin peripheral airway branches can be aligned. The SyN step `grad_step` is unrelated to the template shape-update step `gradient_step` of Section “Groupwise template construction with SyGN”, although both equal 0.2.

Similarity metric. Mutual information (MI) measures the statistical dependence between the intensity distributions of the fixed and warped moving image, estimated from their joint histogram. It is *not* the metric used in either reference publication. Avants et al. (2008) developed SyN with *local cross-correlation* (CC), arguing that MI is a global measure well suited to rigid registration but potentially limiting for deformable registration, and that a localised MI estimate becomes unreliable because joint probabilities need many samples; local CC depends only on local means and variances, which can be estimated from few samples. SyGN (Avants et al., 2010) likewise uses local CC both for the registrations and for the template appearance. With the mean-subtracted images $\bar{I} = I - \mu_I$ and $\bar{J} = J - \mu_J$, the means being computed in a small cubic neighbourhood of $\mathbf{x}$, local CC is

$$CC(\bar{I}, \bar{J}, \mathbf{x}) = \frac{\langle \bar{I}, \bar{J} \rangle^2}{\langle \bar{I}, \bar{I} \rangle \langle \bar{J}, \bar{J} \rangle}, \quad (6)$$

with inner products over the same neighbourhood (5^3 voxels in Avants et al. (2008), radius 4 in Avants et al. (2010)). Because the airway volumes are segmentations, any metric compares the spatial patterns of the label values rather than anatomical intensities.

Registration as optimisation. Deformable registration is often written as

$$\hat{\Phi} = \arg \min_{\Phi} D(T, I \circ \Phi) + \lambda R(\Phi), \quad (7)$$

with a dissimilarity term D , a regulariser R and a weight λ . This is the classical elastic formulation. SyN departs from it: Avants et al. (2008) restrict the search to diffeomorphisms and optimise the similarity term essentially without a competing penalty. Regularity comes from the diffeomorphic path structure and from smoothing of the velocity field; the regularisation term contributes little to the total energy but ensures that a path of minimal length is found.

Diffeomorphisms generated by velocity fields. A diffeomorphism is a differentiable map with a differentiable inverse. In SyN, as in other large-deformation methods, it is generated by integrating a time-dependent velocity field $\mathbf{v}(\mathbf{x}, t)$:

$$\frac{\partial \phi(\mathbf{x}, t)}{\partial t} = \mathbf{v}(\phi(\mathbf{x}, t), t), \quad \phi(\mathbf{x}, 0) = \mathbf{x}. \quad (8)$$

The regularity of $\mathbf{v}$ is measured by a norm $\|\mathbf{v}\|_L$ induced by a linear differential operator such as $L = a\nabla^2 + b\text{Id}$, and the path length

$$D(\phi(\cdot, 0), \phi(\cdot, 1)) = \int_0^1 \|\mathbf{v}(\cdot, t)\|_L dt \quad (9)$$

defines a distance between shapes; its infimum over admissible velocity fields is a true metric, whose shortest paths are geodesics. This distance is what SyGN later minimises to find the average shape (Section “Groupwise template construction with SyGN”).

Symmetric two-part formulation. SyN splits the path between two images I and J into two halves. Two diffeomorphisms, ϕ_1 acting on I and ϕ_2 acting on J , are generated by Eq. (8) with their own velocity fields, each integrated from the identity only over $t \in [0, 0.5]$. The two images are deformed towards each other and compared at the midpoint (figure 2):

$$E_S(I, J) = \inf_{\phi_1} \inf_{\phi_2} \int_0^{0.5} \left(\|\mathbf{v}_1(\cdot, t)\|_L^2 + \|\mathbf{v}_2(\cdot, t)\|_L^2 \right) dt + \int_{\Omega} \Pi(I \circ \phi_1(\cdot, 0.5), J \circ \phi_2(\cdot, 0.5)) d\Omega, \quad (10)$$

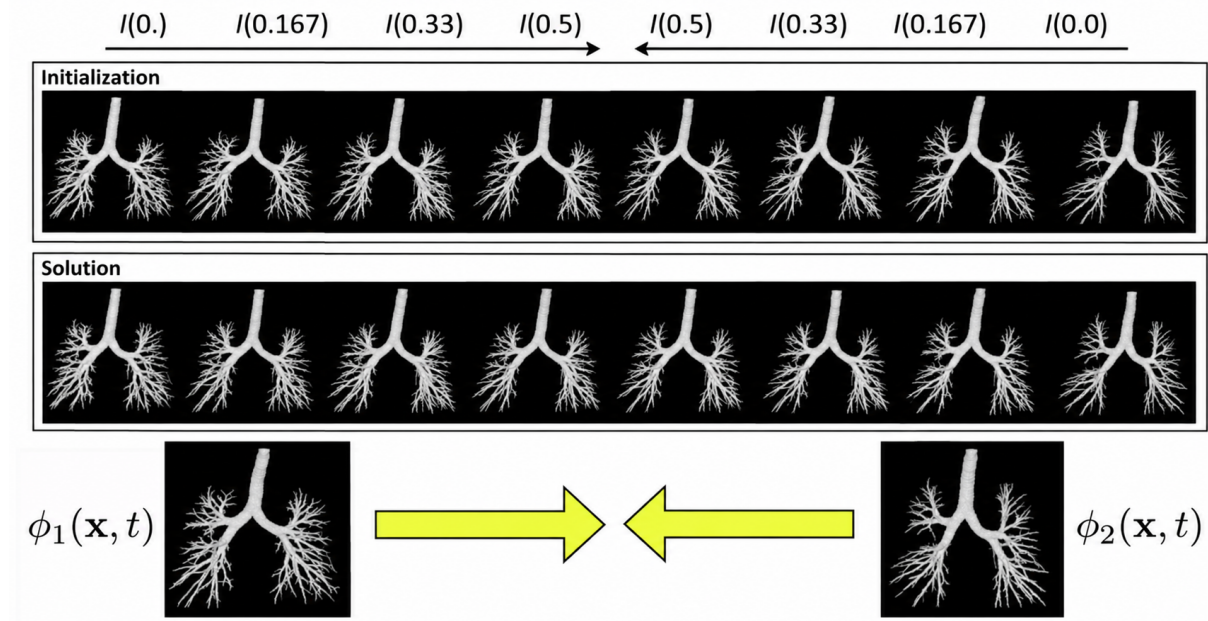


Figure 2: Schematic of the symmetric SyN path between two airway trees I (left) and J (right). Top: the two images at initialisation. Middle: after convergence, both images are deformed along ϕ_1 and ϕ_2 and meet at the mean shape at $t = 0.5$. The full map from I to J is obtained by joining the two half-paths (Eq. 11). Illustrative only; not computed from the study data.

where Π is a pointwise dissimilarity (the squared intensity difference in the derivation of Avants et al. (2008), $-CC$ in their main method). The two half-paths are kept at equal length, so both images travel the same distance and meet at the *mean shape* between them; SyN thus solves a two-image mean problem, the pairwise counterpart of the template estimation in Section “Groupwise template construction with SyGN”.

Full transformations by composition. The complete mapping between the two images is obtained by joining the half-paths:

$$\phi_1(\mathbf{x}, 1) = \phi_2^{-1}(\phi_1(\mathbf{x}, 0.5), 0.5), \quad \phi_1^{-1}(\mathbf{x}, 1) = \phi_1^{-1}(\phi_2(\mathbf{x}, 0.5), 0.5). \quad (11)$$

Because I and J enter Eq. (10) in the same way, exchanging them yields the same path in the opposite direction: the result does not depend on which image is called “fixed” and which “moving”.

Explicit, accurate inverses. Diffeomorphisms are invertible in the continuous setting, but interpolation errors in a discrete implementation accumulate and can break invertibility. SyN therefore computes

the inverses ϕ_i^{-1} explicitly with a fixed-point algorithm and enforces $\phi_i^{-1}(\phi_i) = \text{Id}$ and $\phi_i(\phi_i^{-1}) = \text{Id}$ to sub-voxel accuracy. This distinguishes SyN from inverse-consistent registration, in which forward and backward maps are estimated separately and made consistent only approximately through penalty terms. Both the subject-to-template and the template-to-subject mappings are therefore available and consistent.

Optimisation in practice. Each SyN iteration deforms both images to the midpoint, computes the gradient of the similarity term with respect to each half-map, smooths it into velocity updates, updates ϕ_1 and ϕ_2 through a discretisation of Eq. (8) with a sub-voxel step, recomputes the inverses and forms the maps of Eq. (11). The operator L is approximated by a Gaussian kernel applied to the *incremental* update rather than to the total displacement (`flow_sigma` and `total_sigma` in ANTsPy); smoothing the increment is what permits large diffeomorphic deformations, whereas smoothing the total displacement, as in elastic and Demons-type methods, restricts the solution to small deformations. The optimisation runs from coarse to fine resolution levels.

III Groupwise template construction with SyGN

Formulation. SyGN seeks the template and the set of transformations that give the “smallest” parameterisation of the dataset, measured by the SyN energy of Eq. (10). The template is described by an appearance $\bar{I}$ (its intensities) and a shape, represented by a diffeomorphism ψ that defines the template’s coordinate system. Given the images $\{J^i\}_{i=1}^N$, Avants et al. (2010) minimise

$$E_{\bar{I}} = \sum_{i=1}^N E_S(\bar{I}, J^i, \phi^i) \quad \text{with} \quad \phi^i(\mathbf{x}, 0) = \psi(\mathbf{x}) \quad \forall i, \quad (12)$$

jointly over the mappings ϕ^i , the appearance $\bar{I}$ and the shape ψ . No initial template has to be chosen by the user, and the stated goals are that the final template’s appearance does not depend on any specific anatomy and its shape does not depend on any individual’s coordinate system.

Coordinate-descent algorithm. Eq. (12) is minimised by coordinate descent, one group of variables at a time with the others fixed. After initialisation, each iteration (1) registers every image to the fixed template with SyN; (2) updates the template appearance $\bar{I}$; (3) updates the template shape ψ towards the average of the mappings; and (4) resamples the template through the new ψ , resets the mappings to start from it, and repeats. The template is only updated after all registrations of an iteration are complete, so appearance and shape are never estimated from partially optimised mappings. `ants.build_template` implements the same loop, but with a different initialisation and a different appearance update, as described next (figure 3).

Initialisation. In Avants et al. (2010), the first template is the voxelwise (Euclidean) average of the images *after affine alignment*, and all ϕ^i and ψ start as the identity. In the present implementation no initial template was supplied, and ANTsPy forms the initial template *without any alignment*: each image is multiplied by its weight $w_i = 1/N$, resampled onto the voxel grid of the first input image I_1 , and summed,

$$T^{(0)}(\mathbf{x}) = \sum_{i=1}^N w_i I_i(\mathbf{x}), \quad \mathbf{x} \in \Omega_{I_1}. \quad (13)$$

Two consequences follow. First, $T^{(0)}$ averages the airway trees as they lie in their own scanner coordinates, so it is blurred and its quality depends on how well these coordinate systems already agree.

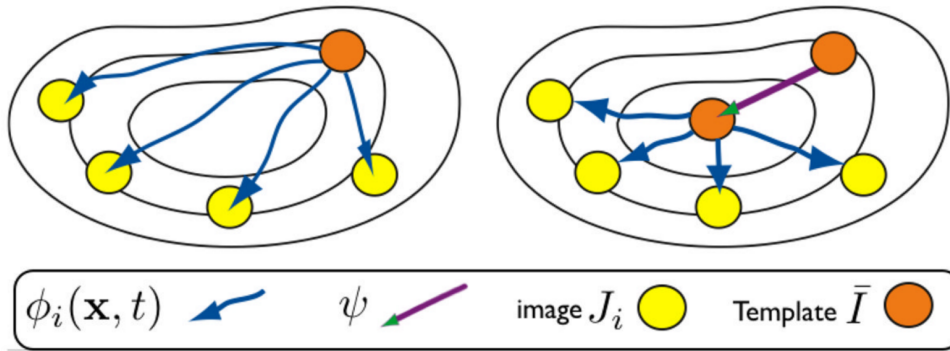


Figure 3: Two steps of one SyGN iteration. Left: the diffeomorphic paths ϕ_i from the current template $\bar{I}$ (orange) to the images J_i (yellow) are estimated (Step 1). Right: the template shape is moved by ψ towards the population centre, which shortens the total path length (Step 3). Reproduced from Avants et al. (2010), Fig. 2.

Because the inputs are binary segmentations, it also contains intermediate values equal to the fraction of volumes that contain airway at each location. Second, every later template is resampled on this grid, so the origin, spacing, orientation and field of view of the final template are inherited from I_1 , here the first randomly selected ATM'22 volume. The template's *shape* is still optimised over the whole population, but its *grid* is not independent of the input order.

Step 1 – registration. With $T^{(k)}$ fixed, each I_i is registered to it with the ANTsPy SyN preset (Section “Pairwise registration with SyN”), giving the affine maps $A_i^{(k)}$, the deformable maps $\phi_i^{(k)}$ and the warped images $I_i \circ \Phi_i^{(k)}$, all on the template grid.

Step 2 – appearance update. Avants et al. (2010) do not take the template appearance to be a plain average. They optimise it by gradient ascent on the cross-correlation of Eq. (6) between the template and the warped images (intensities normalised to $[0, 1]$, gradients smoothed with a Gaussian of $\sigma = 1$ voxel, step size 0.1), and show that this preserves more contrast and detail than the Euclidean mean. ANTsPy does *not* implement this step. Its appearance is the weighted Euclidean mean of the warped images,

$$T_{\text{avg}}^{(k+1)}(\mathbf{x}) = \sum_{i=1}^N w_i \left[I_i \circ \Phi_i^{(k)} \right](\mathbf{x}), \quad (14)$$

and, at the end of each iteration (after the shape update below), this image is blended with a sharpened copy of itself,

$$T^{(k+1)} = \beta T_{\text{shape}}^{(k+1)} + (1 - \beta) \text{Sharpen}(T_{\text{shape}}^{(k+1)}), \quad (15)$$

with $\beta = \text{blending_weight} = 0.75$. The sharpening blend is a heuristic of the implementation that compensates for the blurring of the mean; it appears in neither publication and does not deform the template. In both the paper and the implementation, averaging happens *after* spatial alignment, so each template location combines corresponding locations of the airway trees rather than identical voxel indices of the original images.

Step 3 – shape update. Averaging aligned images fixes the appearance but not the *position* of the template within the population: if the current template is closer to some subjects than to others, the

mappings to those subjects are short and the others long (figure 3, right). Avants et al. (2010) therefore move the template towards the Fréchet (intrinsic) mean of the diffeomorphisms, the shape ψ that minimises the total squared geodesic distance $\sum_i D^2(\psi, \phi^i(\cdot, 1))$ of Eq. (9). Velocity fields, unlike diffeomorphisms, can be added, so the average can be taken over the initial velocity fields of the mappings; setting the derivative of the total distance to zero gives the optimality condition

$$\bar{\mathbf{v}} = \sum_{i=1}^N \mathbf{v}^i(\psi, 0) = 0, \quad (16)$$

and, away from the optimum, the template shape evolves along $d\psi/d\bar{t} = \bar{\mathbf{v}}(\psi, \bar{t})$. Under a small-deformation approximation, with $\psi = \text{Id}$ at the current template, one step of length $\Delta\bar{t}$ becomes a scaled sum of the displacements of the mappings:

$$\psi_{\Delta\bar{t}}(\mathbf{x}) = \mathbf{x} + \frac{\Delta\bar{t}}{\Delta t} \sum_{i=1}^N (\phi_1^i(\mathbf{x}, \Delta t) - \mathbf{x}). \quad (17)$$

At the average configuration the displacements cancel and the template no longer moves. ANTsPy implements this approximation: it averages the forward displacement fields $W_i^{(k)}$ of the deformable registrations, $W^{(k)} = \sum_i w_i W_i^{(k)}$, scales the result by $-\alpha$ with $\alpha = \text{gradient_step} = 0.2$ (the counterpart of $\Delta\bar{t}$), combines it with the inverse of the average affine transformation $\bar{A}^{(k)}$ of the registrations, and resamples the averaged image through the resulting map $\Psi^{(k)}$:

$$T_{\text{shape}}^{(k+1)} = T_{\text{avg}}^{(k+1)} \circ \Psi^{(k)}, \quad \Psi^{(k)} \text{ built from } -\alpha W^{(k)} \text{ and } (\bar{A}^{(k)})^{-1}. \quad (18)$$

Two simplifications relative to the paper should be noted: ANTsPy averages total displacement fields rather than initial velocity fields, and it also re-centres the template affinely. With `useNoRigid=True` (the default), the rigid part is excluded from $\bar{A}^{(k)}$, so only the average scaling and shear are removed and the template does not drift by an arbitrary common rotation or translation. Eq. (18) is schematic; the exact composition order follows the ANTsPy source.

Why the shape update matters for segmentation volumes. Avants et al. (2010) illustrate this step (Figure 4) with a synthetic set of four binary ellipses with the same centre and different radii. The initial average has intermediate grey levels, like the averaged airway masks here. With the shape update, SyGN converges, up to interpolation error, to the true mean shape. Without it, optimising appearance alone converges to a geometrically wrong template, the 0.5 level set of the initial average. The shape update is therefore what makes the geometry of a template built from binary segmentations reflect the population average rather than the initialisation; setting `gradient_step > 0` is essential for this.

Iterations and convergence. The loop was run for a fixed number of 8 iterations (the ANTsPy default is 3). Avants et al. (2010) report that SyGN typically converges in fewer than ten iterations, usually three to five, depending on the complexity of the deformations. No stopping criterion is applied by the code. However, ANTsPy prints the mean absolute value of the average displacement field $W^{(k)}$ at every iteration; by Eq. (16) this quantity should tend towards zero as the template approaches the population mean, and it can be used to assess convergence.

Unbiasedness and its limits. In principle SyGN gives a template that is unbiased: it requires no user-selected reference, uses a symmetric pairwise method, and centres the template shape on the population. In the present implementation this holds for the template shape up to three qualifications:

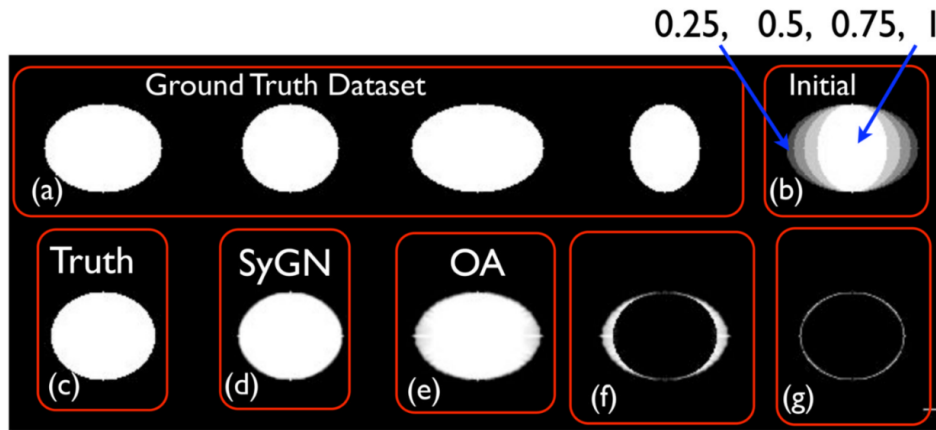


Figure 4: Effect of the shape update on binary images. (a) Four binary ellipses; (b) their initial average, with grey levels 0.25–1; (c) true mean shape; (d) SyGN result with shape update; (e) result without shape update (appearance only), which converges to the wrong shape; (f,g) errors of (e) and (d) with respect to (c). Reproduced from Avants et al. (2010), Fig. 3.

the template grid is inherited from the first input volume (Eq. (13)); the eight volumes are a small random subset of the two datasets; and the registrations and the appearance use MI and a sharpened mean instead of the cross-correlation model of the reference publications.

Output. After the eighth iteration, the template $T^{(8)}$ was written to `template.nii.gz`. It is the reference used in Section “Affine registration”. The complete procedure is summarised in Figure 5.

The NIfTI geometry defines where each original voxel grid lies in physical space, whereas the affine and diffeomorphic transformations estimated by ANTs establish anatomical correspondence between these physical domains. The final template is therefore a population-derived coordinate space in which the anatomical variability of the eight airway trees has been brought into correspondence.

II Affine registration

Airway trees extracted from different patients differ in position, orientation and overall size, as a consequence of patient positioning in the scanner, differences in body size and acquisition parameters. To make them directly comparable, all airway segmentations were mapped into the constructed template. An affine transformation was chosen for this purpose because it compensates for these global differences while preserving the local geometry of each airway tree, which is the object of the subsequent analysis.

Affine registration consists of finding the spatial transformation T that best aligns a *moving* image M with a *fixed* image F . In this work, the fixed image is the template and each AIB23 segmentation is, in turn, the moving image. The transformation is restricted to the class of three-dimensional affine maps,

$$T_{\mu}(\mathbf{x}) = \mathbf{A}(\mathbf{x} - \mathbf{c}) + \mathbf{c} + \mathbf{t}, \quad (19)$$

where $\mathbf{x} \in \mathbb{R}^3$ is a point in the fixed image domain, $\mathbf{A} \in \mathbb{R}^{3 \times 3}$ is a general matrix, $\mathbf{t} \in \mathbb{R}^3$ is a translation vector and $\mathbf{c}$ is a fixed centre of rotation. The parameter vector μ therefore contains $9 + 3 = 12$ degrees of freedom, which jointly encode translation, rotation, anisotropic scaling and shear. Because the transformation is linear up to a translation, straight lines are mapped to straight lines and parallelism is preserved, so the shape of the airway tree is modified only globally.

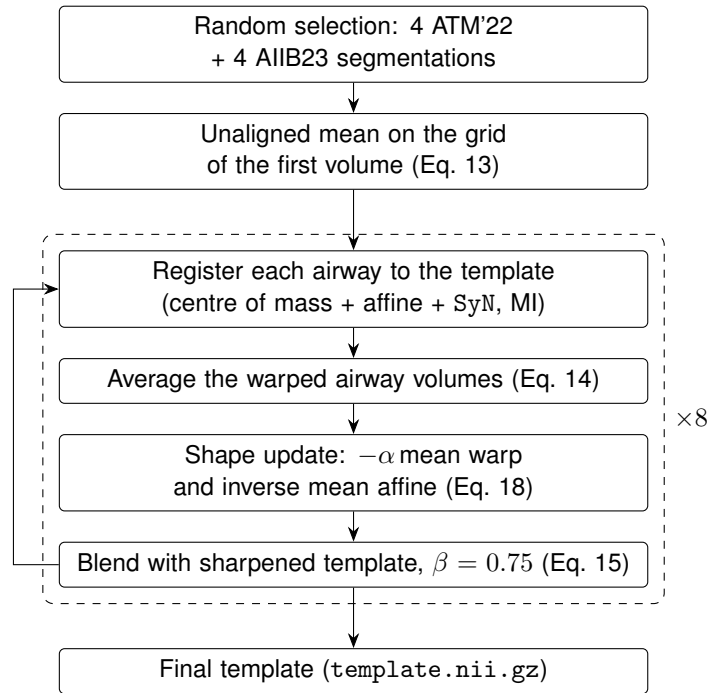


Figure 5: Groupwise template construction with `ants.build_template`. The dashed block is one SyGN iteration, repeated eight times.

Similarity metric and optimisation. The optimal parameters are obtained by solving

$$\hat{\mu} = \arg \min_{\mu} \mathcal{C}(F, M \circ T_{\mu}), \quad (20)$$

where $\mathcal{C}$ is a cost function measuring the dissimilarity between the fixed image and the transformed moving image. The cost function used is the negative of the Mattes mutual information (Mattes et al., 2003),

$$\mathcal{C}(\mu) = - \sum_{\iota} \sum_{\kappa} p(\iota, \kappa; \mu) \log \frac{p(\iota, \kappa; \mu)}{p_F(\iota) p_M(\kappa; \mu)}, \quad (21)$$

where $p(\iota, \kappa; \mu)$ is the joint probability of intensity bins ι and κ in the fixed and transformed moving images, and p_F and p_M are the corresponding marginal distributions. In the Mattes formulation, these distributions are estimated with B-spline Parzen windows, which makes the metric differentiable with respect to μ . The histograms were built with 32 bins from a random sample of 20% of the voxels, which reduces the computational cost of each iteration.

Equation (20) was solved with a gradient descent optimiser. Before optimisation, the transformation was initialised by aligning the intensity centres of mass of the two images, which provides a coarse translational alignment and reduces the risk of convergence to a poor local optimum.

Multi-resolution strategy. To improve robustness and speed, the optimisation was performed within a four-level multi-resolution scheme (Thévenaz et al., 1998). At each level, both images were smoothed with a Gaussian kernel and downsampled by an integer shrink factor; the optimisation then proceeded from the coarsest level to the original resolution, with the result of each level used to initialise the next. Coarse levels capture the large-scale alignment, while finer levels refine it. The settings of each level are listed in Table 5.

Table 5: Multi-resolution settings of the affine registration.

Level	Shrink factor	Gaussian σ (voxels)	Maximum iterations
1 (coarsest)	6	3	2100
2	4	2	1200
3	2	1	1200
4 (finest)	1	0	10

At each level, the optimisation stopped either when the maximum number of iterations was reached or when the cost function had converged, i.e. when its variation over the last 10 iterations fell below 10^{-6} .

Resampling of the registered images. Once the optimal transformation $T_{\hat{\mu}}$ had been estimated, each moving segmentation was resampled onto the voxel grid of the template using linear interpolation and saved in NIfTI format. Since the inputs are binary masks, linear interpolation yields intermediate values in $[0, 1]$ at the boundary of the airway lumen, while voxels well inside or outside the airway keep the values 1 and 0 respectively.

I Encoding

[Optional short introduction: why a learned representation of the airway trees is needed, and the three approaches tried – a variational autoencoder, deterministic and sparse autoencoders, and an implicit encoding.]

II Variational Autoencoder for Airway Tree Representation Learning

III Motivation

To learn a compact representation of airway trees, we represent each tree as a binary voxel occupancy grid. As Brock et al. (2016) point out, there is no meaningful way to interpolate directly between two binary grids. Relating voxelized shapes to one another therefore requires a model that works in an abstract feature space capturing the main factors of variation.

For this reason we use a Variational Autoencoder (VAE) (Kingma and Welling, 2014). A VAE is a probabilistic framework that learns two networks jointly:

- an **inference (encoder) network**, which maps an input to a set of latent variables describing it, and
- a **generative (decoder) network**, which maps from the latent space back to the input space.

Once the encoder has learned latent variables that describe the underlying variation between shapes, each airway tree can be encoded as a latent vector. That vector can be used as a learned representation, or interpolated and decoded back into a voxel grid. Our models are adapted from the voxel-based VAE of Brock et al. (2016).

III Model architecture

Input resolutions. Each airway tree was voxelized into a binary occupancy grid at two resolutions, $128 \times 128 \times 128$ and $256 \times 256 \times 256$ voxels. The higher resolution preserves more of the thin distal branches, at the cost of a much larger memory and computational footprint. We trained a separate VAE for each resolution. Both networks follow the design principles of Brock et al. (2016), adapted to the larger input size. The 256^3 model has one additional downsampling stage (two extra encoder layers), so that both networks reach the same $7 \times 7 \times 7$ spatial grid at the bottleneck. The architectures are summarized in Figure 6 and Table 6.

Encoder. The encoder is a stack of 3D convolutional blocks. Each block consists of a $3 \times 3 \times 3$ convolution without bias, instance normalization with learnable affine parameters, and an ELU activation. As in Brock et al. (2016), downsampling is performed with strided convolutions rather than pooling, in every second layer. The layers alternate between an unpadded stride-1 convolution, which reduces each spatial dimension by 2, and a stride-2 convolution with padding 1, which approximately halves it. The number of filters doubles at every layer:

- **128^3 model:** 8 convolutional layers, starting with 2 filters and reaching 256 filters. The spatial size goes $128 \rightarrow 126 \rightarrow 63 \rightarrow 61 \rightarrow 31 \rightarrow 29 \rightarrow 15 \rightarrow 13 \rightarrow 7$.
- **256^3 model:** 10 convolutional layers, starting with 4 filters and reaching 2048 filters. The spatial size goes $256 \rightarrow 254 \rightarrow 127 \rightarrow 125 \rightarrow 63 \rightarrow 61 \rightarrow 31 \rightarrow 29 \rightarrow 15 \rightarrow 13 \rightarrow 7$.

A $1 \times 1 \times 1$ convolution (followed by instance normalization and ELU) then reduces the number of channels to 32 while keeping the 7^3 grid. The result is flattened into a vector of $32 \cdot 7^3 = 10,976$ features. In Brock et al. (2016), a fully connected layer is placed between the convolutional features and the latent projection. In our models this layer is removed, and the flattened features are mapped directly by two separate linear layers to the mean μ and the log standard deviation $\log \sigma$ of the latent Gaussian, of dimension [TODO: 100]. The log standard deviation is clamped to $[-10, 10]$ for numerical stability.

Latent sampling. During training, a latent vector is sampled with the reparameterization trick (Kingma and Welling, 2014),

$$\mathbf{z} = \mu + \exp(\log \sigma) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (22)$$

At inference time the mean is used directly ($\mathbf{z} = \mu$), giving a deterministic encoding of each airway tree.

Decoder. The decoder mirrors the encoder, and its weights are not tied to the encoder's. A linear layer maps $\mathbf{z}$ back to 10,976 features, followed by layer normalization (Ba et al., 2016) and ELU, and the result is reshaped into a 32×7^3 volume. A stride-1 convolutional block expands the channels to the width of the deepest encoder layer (256 for the 128^3 model, 2048 for the 256^3 model). The decoder then alternates between upsampling blocks and stride-1 convolutional blocks (padding 1, spatial size preserved), halving the number of channels at every layer.

Brock et al. (2016) upsample with fractionally strided (transposed) convolutions. We instead use a resize-convolution: each upsampling block applies nearest-neighbour upsampling to an exact target size, followed by a $3 \times 3 \times 3$ convolution, instance normalization and ELU. This avoids the checkerboard artifacts associated with transposed convolutions (Odena et al., 2016) and lets the decoder match the

odd intermediate sizes of the encoder (15, 31, 63, 127) exactly. The spatial size goes $7 \rightarrow 15 \rightarrow 31 \rightarrow 63 \rightarrow 128$ for the 128^3 model and $7 \rightarrow 15 \rightarrow 31 \rightarrow 63 \rightarrow 127 \rightarrow 256$ for the 256^3 model.

A final $3 \times 3 \times 3$ convolution with bias maps the features to a single output channel, with no normalization or activation. Its outputs are logits; the sigmoid applied inside the loss converts them into the predicted probability that each voxel is occupied.

Normalization, activations and initialization. All layers use the ELU activation (Clevert et al., 2016), except the output layer. Where Brock et al. use Batch Normalization (Ioffe and Szegedy, 2015), we use instance normalization (Ulyanov et al., 2016). At 128^3 and 256^3 resolution, GPU memory limits the batch size to a few samples, which makes batch statistics unreliable; instance normalization normalizes each sample independently and does not depend on batch size. Convolutional and linear weights are initialized with Glorot (Xavier) normal initialization (Glorot and Bengio, 2010), and biases are initialized to zero.

Table 6: Summary of the two VAE architectures.

Stage	128^3 model	256^3 model
Input	1×128^3	1×256^3
Encoder conv layers	8	10
Filters (first $\rightarrow$ last)	$2 \rightarrow 256$	$4 \rightarrow 2048$
Encoder output	256×7^3	2048×7^3
$1 \times 1 \times 1$ reduction	32×7^3	32×7^3
Flattened features	10,976	10,976
Latent dimension	[TODO: 100]	[TODO: 100]
Decoder upsampling steps	4	5
Decoder filters (first $\rightarrow$ last)	$256 \rightarrow 2$	$2048 \rightarrow 4$
Output	1×128^3 (logits)	1×256^3 (logits)

[VAE architecture figure]

Figure 6: Architecture of the voxel-based VAEs, adapted from Brock et al. (2016). Shown for the 256^3 model; the 128^3 model omits the first downsampling stage.

Loss function

Following Brock et al. (2016), the loss combines three terms: the KL divergence between the approximate posterior and the prior on the latent variables, L2 weight regularization, and a reconstruction term based on a modified Binary Cross-Entropy (BCE). We then add a fourth term, cIDice, described below.

Modified BCE. The standard BCE is

$$\mathcal{L}_{\text{BCE}} = -t \log(o) - (1 - t) \log(1 - o), \quad (23)$$

where $t \in \{0, 1\}$ is the target voxel value and $o \in (0, 1)$ is the network output. Brock et al. (2016) identify two problems with this loss for voxel data. First, its derivative with respect to o becomes very small as o approaches t , which can cause vanishing gradients. Second, it penalizes false positives and false negatives equally. Most of a voxel grid is empty, so the network can fall into a poor local optimum by predicting every voxel as empty. The second problem is even more severe for airway trees than for the ModelNet objects in the original paper: airways are thin, branching structures that occupy a very small fraction of the grid.

Brock et al. (2016) propose two modifications to address these problems: shifting the target and output ranges to increase the gradient magnitude, and weighting false negatives against false positives. In our implementation we adopt the second modification. The targets remain binary, $t \in \{0, 1\}$, and the output o is the sigmoid of the decoder logits, clamped to $[\epsilon, 1 - \epsilon]$ with $\epsilon = 10^{-7}$ to avoid $\log(0)$. A hyperparameter γ weights false negatives against false positives:

$$\mathcal{L}_{\text{BCE-mod}} = -\gamma t \log(o) - (1 - \gamma)(1 - t) \log(1 - o). \quad (24)$$

The loss is summed over all voxels of a sample and averaged over the batch. We set $\gamma = \text{[TODO: 0.99]}$, which strongly penalizes missed voxels and reduces the penalty for spurious ones; this is slightly higher than the 0.97 used by Brock et al., reflecting the even stronger class imbalance of airway trees. Brock et al. report that a γ that is too high gives noisy reconstructions, while a γ that is too low gives reconstructions that lose important structural details.

KL term. The KL divergence is summed over the latent dimensions and averaged over the batch, and it is weighted by a factor β , following the β -VAE formulation (Higgins et al., 2017). L2 weight regularization is applied through the optimizer’s weight decay, with coefficient **[TODO: value]**.

clDice (our addition). Voxel-wise losses do not explicitly preserve the connectivity of tubular structures, which is essential for airway trees. We therefore add the centerline Dice (clDice) loss (Shit et al., 2021) to the reconstruction objective. Let V_P and V_L be the predicted and ground-truth volumes, and S_P and S_L their soft skeletons. clDice is defined as

$$T_{\text{prec}} = \frac{|S_P \cap V_L|}{|S_P|}, \quad T_{\text{sens}} = \frac{|S_L \cap V_P|}{|S_L|}, \quad (25)$$

$$\text{clDice} = 2 \frac{T_{\text{prec}} \cdot T_{\text{sens}}}{T_{\text{prec}} + T_{\text{sens}}}. \quad (26)$$

Topology precision is the fraction of the predicted skeleton lying inside the ground-truth volume, and topology sensitivity is the fraction of the ground-truth skeleton lying inside the predicted volume. The final training objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE-mod}} + \beta \mathcal{L}_{\text{KL}} + \lambda (1 - \text{clDice}) + \mathcal{L}_{\text{L2}}, \quad (27)$$

with $\beta = \text{[TODO: value]}$ and $\lambda = \text{[TODO: value]}$.

III Training

The model is trained with stochastic gradient descent using Nesterov momentum (Sutskever et al., 2013). Training runs for up to [TODO: 100] epochs, or until the reconstruction error on a held-out validation set stops decreasing. Following Brock et al. (2016), the learning rate is [TODO: 0.0001] for the first epoch and then raised to [TODO: 0.001]. The dataset contains [TODO: N] airway trees, split into [TODO: N_{train}] training, [TODO: N_{val}] validation and [TODO: N_{test}] test samples. Training used a batch size of [TODO: batch size] on [TODO: GPU].

III Results

During training, the reconstruction loss on the training set decreased steadily, and the model reconstructed training airway trees well. Performance on held-out data was much worse. The validation loss did not follow the training loss (Figure 7). Reconstructions of validation and test trees were poor, with [TODO: e.g. missing branches, broken connectivity, blurred or blob-like structures] (Figure 8).

This gap between training and validation performance shows that the model **overfits**: it memorizes the training samples instead of learning a representation that generalizes to unseen airway trees.

Brock et al. (2016) observed a relevant limitation of their own voxel VAE. It struggled to reconstruct objects with long, thin parts, such as table and chair legs. The authors suggest that such small features contribute little to the global loss and get lost in favor of denser regions. They also note that the VAE tends to produce smooth, rounded outputs rather than sharp edges. Airway trees consist almost entirely of thin, branching tubes, so this limitation likely contributes to the poor reconstructions we observed, even with the added cIDice term.

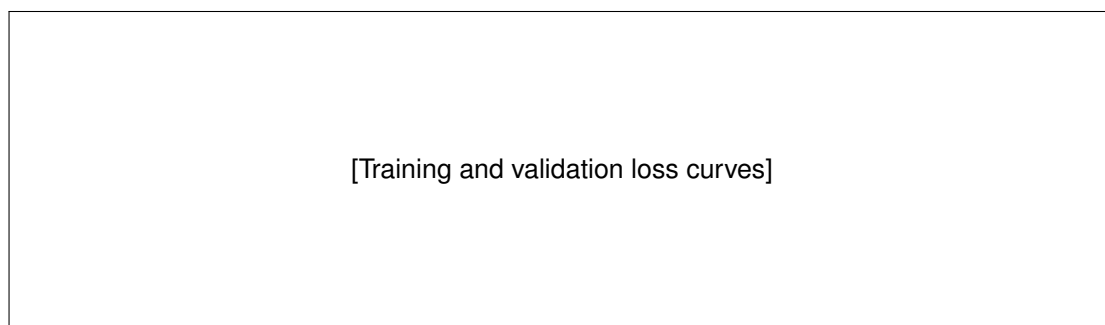


Figure 7: Training and validation losses of the VAE. The validation loss does not follow the decrease of the training loss, indicating overfitting.

III Mitigating overfitting

To reduce overfitting, we applied two regularization strategies.

Data augmentation. Each training sample was randomly transformed with [TODO: random translations, flips, rotations, small elastic deformations, noise]. This increases the effective size and variability of the training set. Brock et al. (2016) use a similar strategy, training on both a corrupted and an uncorrupted copy of each instance so that the network learns invariance to small structural variations.

[Original vs. reconstructed airway trees]

Figure 8: Original airway trees and their VAE reconstructions for training (top) and validation/test (bottom) samples.

Dropout. Dropout layers with rate **[TODO: p]** were added to **[TODO: the fully connected layers / the encoder / the decoder]** to discourage co-adaptation of features.

Neither strategy resolved the problem. As shown in Figure 9, the training and validation curves still diverge after adding augmentation and dropout, and validation and test reconstructions remain poor. This suggests the limitation lies deeper than a lack of regularization. Plausible causes include the small size of the dataset relative to model capacity, the low voxel resolution relative to the thinness of the airway branches, and the difficulty voxel VAEs have with thin structures.

[Loss curves with augmentation and dropout]

Figure 9: Training and validation losses after adding data augmentation and dropout. The gap between training and validation performance persists.

II Deterministic and Sparse Autoencoders

In addition to the VAEs described in Section “Variational autoencoder”, we trained two further variants at both input resolutions (128^3 and 256^3): a deterministic autoencoder, and a model with a sparse convolutional encoder.

III Deterministic autoencoder

The deterministic autoencoder uses the same encoder and decoder as the VAE (Section “Model architecture”), with the variational part removed. Instead of two linear layers predicting μ and $\log \sigma$, a single linear layer maps the flattened encoder features directly to a latent code z of dimension **[TODO: 100]**. No sampling takes place, and the KL divergence term is dropped from the loss, so the objective reduces to the weighted BCE, the cIDice term and L2 weight regularization:

$$\mathcal{L}_{AE} = \mathcal{L}_{BCE-mod} + \lambda (1 - cIDice) + \mathcal{L}_{L2}. \quad (28)$$

Without the KL term, the latent space is no longer pushed towards a unit Gaussian prior. This means the model cannot be used to generate new trees by sampling, but the latent code can still serve as a learned representation of each airway tree. Removing the KL term also removes the trade-off between reconstruction quality and latent regularization. This lets us test whether the KL term was responsible for the poor reconstructions of the VAE.

III Sparse encoder

Motivation. The second variant replaces the dense convolutional encoder with a sparse one, following the sparse structure VAE of XCube (Ren et al., 2024). Ren et al. argue that voxel grids are flexible, can represent both bulky and thin structures, and allow fast querying and processing. Sparse voxel representations go further by not storing any information for empty regions, which makes them considerably more efficient (Ren et al., 2024). This is particularly relevant for airway trees: only a very small fraction of the 128^3 or 256^3 grid is occupied, so a dense encoder spends most of its computation and memory on empty space.

Encoder design. As in XCube (Ren et al., 2024), the encoder is built from neural network operators defined over sparse voxel grids. It processes the input occupancy grid with a sparse convolutional neural network that alternates between sparse convolutions and max pooling operations, reducing the resolution step by step until it reaches the resolution of the latent representation. XCube implements these operators on top of the VDB data structure (Museth, 2013), whose compact storage and fast look-up make sparse convolution and pooling efficient on the GPU. We implemented the sparse encoder with [TODO: sparse convolution library, e.g. fVDB / MinkowskiEngine / spconv], using [TODO: number] sparse convolution layers and [TODO: number] max pooling layers. The resulting sparse features are then [TODO: converted to a dense 7^3 grid / globally pooled] and mapped to the latent code in the same way as in the dense model.

Decoder. We kept the decoder unchanged: it is the same dense decoder described in Section “Model architecture”. Note that XCube pairs its sparse encoder with a structure-prediction decoder, which starts from the latent grid and progressively subdivides existing voxels and prunes unnecessary ones until it reaches the output resolution (Ren et al., 2024). We did not adopt this decoder, so that any change in performance could be attributed to the encoder alone. The sparse-encoder model was trained with the [TODO: VAE objective of Section “Loss function” / autoencoder objective of Equation (28)].

III Results

Neither architecture produced satisfactory reconstructions. As with the VAE, reconstructions of validation and test airway trees show [TODO: e.g. missing branches, broken connectivity, blob-like structures] (Figure 10).

Both models also show the same overfitting behaviour as the VAE. The training loss decreases steadily, while the validation loss does not follow it (Figure 11). [TODO: Data augmentation and dropout were also applied to these models, without improvement.]

The deterministic autoencoder, the sparse-encoder model and the VAE all show the same pattern. This suggests that the problem is not caused by the variational formulation or by the type of encoder. More likely causes are the limited amount of training data relative to model capacity and the difficulty of reconstructing thin, branching structures with a dense voxel decoder.

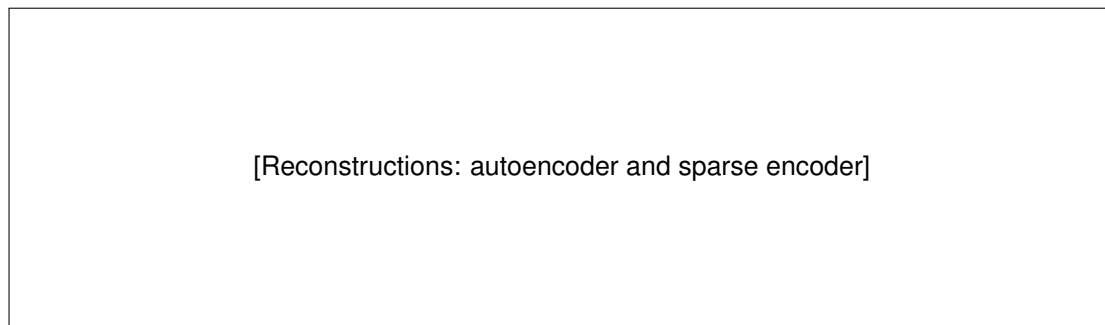


Figure 10: Original airway trees and their reconstructions from (a) the deterministic autoencoder and (b) the sparse-encoder model, for training and validation/test samples.

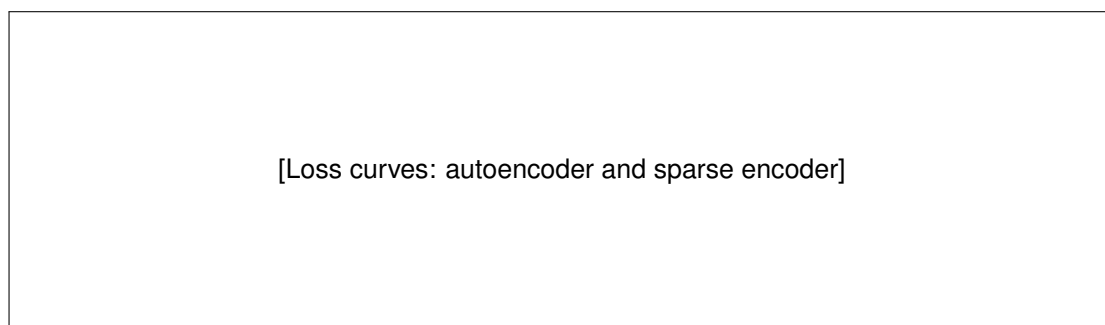


Figure 11: Training and validation losses of (a) the deterministic autoencoder and (b) the sparse-encoder model. As with the VAE, the validation loss does not follow the training loss.

II Implicit Neural Representation of Airway Trees

The voxel-based models of Section “Variational autoencoder” and Section “Deterministic and sparse autoencoders” did not produce satisfactory reconstructions. We therefore trained an implicit model, the Deep Geometric Correspondence Implicit (DGCI) network of Zhang et al. (2024). This model gave the best reconstructions of all the architectures we tested, and its learned latent codes are the representation used for the clustering in Section “Clustering”.

III Motivation

Zhang et al. (2024) point out that airway annotations are made slice by slice in CT image stacks, so the reconstructed shape is inevitably limited by the resolution of the discrete voxels. The image contrast also degrades as the bronchi become thinner. As a result, voxel-based airway trees tend to be coarse and affected by spike-like noise, and CNN-based methods, which also work in the discrete voxel space, suffer from the same problems as well as from breakages in the tree (Zhang et al., 2024).

Implicit neural representations (INRs) address this by using coordinate-based neural networks to parameterize a shape in continuous physical space instead of on a voxel grid. They are therefore not bounded by the CT grid resolution, and they yield smoother, high-fidelity shapes with fewer spike-like artifacts (Zhang et al., 2024). Earlier INRs such as DeepSDF (Park et al., 2019) focus only on surface reconstruction and do not capture topological correspondence between shapes of the same category. DGCI is built on the hypothesis that all airway trees share a common topology, since they are all single connected tree structures, and it uses this shared structure to constrain the learned feature space of the training shapes (Zhang et al., 2024).

III Model architecture

Neural signed distance field. Following DeepSDF (Park et al., 2019), each airway tree is represented as a signed distance field (SDF) in continuous space. A neural field function $\mathcal{F}$ takes a physical coordinate $\mathbf{p} \in \mathbb{R}^3$ together with an instance-specific latent code $\alpha_i \in \mathbb{R}^K$ and predicts the signed distance s of that point to the surface of the i -th airway tree:

$$\mathcal{F}(\mathbf{p}; \alpha_i) : \mathbb{R}^{3+K} \rightarrow s \in \mathbb{R}. \quad (29)$$

The airway surface is the zero level set of $\mathcal{F}$.

Deep correspondence flow. To model airways in continuous space while also exploiting the topological features shared across cases, DGCI builds a reversible correspondence flow between an *instance space*, specific to each airway tree, and a *shared space*, common to all trees (Figure 12). This flow is implemented by an implicit encoder–decoder, called DGCI-ED, in which both the encoder E and the decoder D are multilayer perceptrons (MLPs) (Zhang et al., 2024). The encoder is driven by a shared latent code β , common to all instances, and the decoder by the instance-specific code α_i .

Hypernetworks. The latent codes do not enter the MLPs as ordinary inputs. Instead, following the hypernetwork design (Ha et al., 2017; Sitzmann et al., 2020), α_i and β drive two separate hypernetworks, ϕ_{α_i} and ϕ_{β} . Each hypernetwork is itself a set of MLPs, and each of these MLPs generates the weights of one fully connected layer of the target network (Zhang et al., 2024). ϕ_{α_i} generates the weights of the decoder D and of the SDF function $\mathcal{F}$; ϕ_{β} generates the weights of the encoder E . The encoder–decoder can thus be written as $\text{DGCI-ED} = D_{\phi_{\alpha_i}} \circ E_{\phi_{\beta}}$.

Forward pass. Since $E_{\phi_{\beta}}$ is controlled by a code shared by every instance, it forces the model to learn the topological features common to all airway trees. It takes the physical coordinates together with the SDF values produced by $\mathcal{F}$ and encodes this geometric information into an intermediate latent feature $\mathbf{z}_i$ (Zhang et al., 2024):

$$E_{\phi_{\beta}}(\mathbf{p}, \mathcal{F}_{\phi_{\alpha_i}}(\mathbf{p})) \rightarrow \mathbf{z}_i \in \mathbb{R}^K. \quad (30)$$

This establishes the correspondence from the instance space to the shared space. The instance-specific decoder $D_{\phi_{\alpha_i}}$ then maps $\mathbf{z}_i$ back to the physical instance space, and $\mathcal{F}$ is applied to obtain the final SDF value. The complete model is

$$\begin{aligned} \mathcal{F}(\text{DGCI-ED}(\mathbf{p}; \alpha_i, \beta)) &= \mathcal{F}_{\phi_{\alpha_i}}(D_{\phi_{\alpha_i}} \circ E_{\phi_{\beta}}(\mathbf{p}, \mathcal{F}_{\phi_{\alpha_i}}(\mathbf{p}))) \\ &\rightarrow s \in \mathbb{R}. \end{aligned} \quad (31)$$

For brevity, we write $\mathcal{F}(\mathbf{p}; \alpha_i, \beta)$ for this full model below.

Network sizes. Following Zhang et al. (2024), the encoder $E_{\phi_{\beta}}$ has [TODO: three] hidden layers, the decoder $D_{\phi_{\alpha_i}}$ has [TODO: two], and the SDF function $\mathcal{F}$ has [TODO: four]. The weights of every fully connected layer in these three networks are generated by hypernetworks with [TODO: three] hidden layers. The latent codes α_i and β , the hidden features and the intermediate features all have dimension $K =$ [TODO: 128].



Figure 12: Architecture of the Deep Geometric Correspondence Implicit (DGCI) network, adapted from Zhang et al. (2024). Coordinates and the instance code α_i pass through $\mathcal{F}$ and the shared encoder E to the intermediate feature $\mathbf{z}_i$, which the instance decoder D and $\mathcal{F}$ map back to an SDF value s .

Loss function

Zhang et al. (2024) note that the L1 loss on SDF values used by DeepSDF (Park et al., 2019) is not enough to model fine airway structures with high fidelity. Following the implicit geometric regularization of Gropp et al. (2020), DGCI adds two geometric constraints: the gradient of $\mathcal{F}$ at surface points should align with the ground-truth surface normals, and the gradient should have unit norm everywhere (the Eikonal constraint). The SDF loss therefore has two parts, $\mathcal{L}_{\text{sdf}} = \mathcal{L}_{\text{sdf-val}} + \mathcal{L}_{\text{sdf-geo}}$, which supervise the values and the geometry of the predicted SDF respectively:

$$\mathcal{L}_{\text{sdf-val}} = \sum_i \left(\omega_s \sum_{\mathbf{p} \in \Omega_i} |\mathcal{F}(\mathbf{p}; \alpha_i, \beta) - s'| + \omega_\varphi \sum_{\mathbf{p} \in \Omega_i \setminus S_i} \varphi(\mathcal{F}(\mathbf{p}; \alpha_i, \beta)) \right), \quad (32)$$

$$\begin{aligned} \mathcal{L}_{\text{sdf-geo}} = & \sum_i \left(\omega_n \sum_{\mathbf{p} \in S_i} (1 - S_{\cos}(\nabla \mathcal{F}(\mathbf{p}; \alpha_i, \beta), \mathbf{n}')) \right. \\ & \left. + \omega_{\text{Eik}} \sum_{\mathbf{p} \in \Omega_i} | \|\nabla \mathcal{F}(\mathbf{p}; \alpha_i, \beta)\|_2 - 1 | \right). \end{aligned} \quad (33)$$

Here Ω_i is the whole space and S_i the surface of the i -th instance, s' and $\mathbf{n}'$ are the ground-truth SDF values and surface normals, and $S_{\cos}$ is the cosine similarity. Equation (32) supervises the SDF regression directly and, through $\varphi(s) = \exp(-\delta|s|)$ with $\delta \gg 1$, penalizes points off the surface whose predicted SDF is close to zero. Equation (33) enforces consistency with the surface normals and constrains the gradient magnitude according to the Eikonal equation (Zhang et al., 2024).

Two regularization terms act on the latent codes, $\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{intra-reg}} + \mathcal{L}_{\text{inter-reg}}$:

$$\mathcal{L}_{\text{intra-reg}} = \omega_{\text{intra}} \left(\sum_i \|\alpha_i\|_2^2 + \|\beta\|_2^2 \right), \quad \mathcal{L}_{\text{inter-reg}} = \omega_{\text{inter}} \sum_i \|\alpha_i - \beta\|_2^2. \quad (34)$$

The intra-regularization constrains the instance codes and the shared code separately, while the inter-regularization keeps each instance code close to the shared code (Zhang et al., 2024). Finally, a correspondence loss ensures that the deep correspondence flow is accurate, i.e. that decoding the intermediate feature returns the original surface coordinates:

$$\mathcal{L}_{\text{cor}} = \sum_i \left(\omega_{\text{cor}} \sum_{\mathbf{p} \in S_i} \|\mathbf{p} - D_{\phi_{\alpha_i}}(\mathbf{z}_i)\|_2^2 \right). \quad (35)$$

The total objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{sdf}} + \mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{cor}}. \quad (36)$$

Following Zhang et al. (2024), we set ω_s , ω_φ , ω_n and ω_{Eik} to [TODO: 3×10^3 , 5×10^2 , 10^2 and 5×10^1], ω_{intra} and ω_{inter} to [TODO: 10^6 and 10^2], and ω_{cor} to [TODO: 5×10^4].

III Data preparation and training

Since the airway annotations are given as 3D volumes, we first extracted each airway surface with Marching Cubes (Lorensen and Cline, 1987) and then prepared normalized SDF samples following DeepSDF (Park et al., 2019), as done by Zhang et al. (2024). Our dataset of [TODO: N] airway trees was split into [TODO: N_{train}] training, [TODO: N_{val}] validation and [TODO: N_{test}] test cases.

In each iteration, [TODO: 4,000] surface points and [TODO: 4,000] free-space points were randomly sampled per shape. The model was trained for [TODO: 500] epochs with the Adam optimizer (Kingma and Ba, 2015), a learning rate of [TODO: 10^{-4}] and a batch size of [TODO: 16], on [TODO: GPU]. [TODO: Describe how latent codes for validation and test cases were obtained, e.g. by optimizing α_i with the network weights frozen.] The training and validation losses are shown in Figure 13.

After training, each airway tree is represented by its instance latent code $\alpha_i \in \mathbb{R}^{\text{[TODO: 128]}}$. These codes are the features used for clustering in Section “Clustering”.

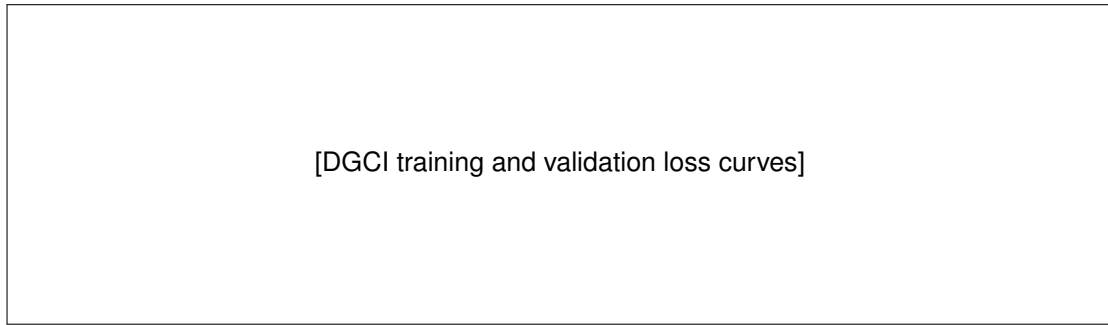


Figure 13: Training and validation losses of the DGCI model: [TODO: total loss and the individual terms $\mathcal{L}_{\text{sdf}}$, $\mathcal{L}_{\text{reg}}$ and $\mathcal{L}_{\text{cor}}$].

III Evaluation metrics

We follow the evaluation protocol of Zhang et al. (2024), which measures shape quality both in continuous space, on the mesh extracted from the predicted SDF, and in discrete voxel space.

Continuous space. Smoothness and connectivity of the reconstructed mesh are measured by counting non-manifold (NM) elements. NM-V counts the vertices connected to three or more non-coplanar faces, NM-E counts the edges shared by three or more faces, and NM-F counts the faces arranged in a way that prevents a closed, continuous body from being formed. Fewer non-manifold elements indicate better connectivity and smoothness (Zhang et al., 2024).

Discrete space. To compute voxel-based metrics, the reconstructed surface is mapped back to the image coordinate system through the inverse of the image-to-physical transform, and a hole-filling algorithm is applied to obtain a dense volume (Zhang et al., 2024). On this volume we compute the tree length detected rate (TD) and the branch detected rate (BD), defined in Eq. (1), and the Dice coefficient, which measure the topological completeness and correctness of the reconstruction (Zhang et al.,

2024).

Results

Table 7 reports the metrics on the [TODO: validation and test] sets, and Figure 14 shows examples of reconstructed airway trees. DGCI produced the best reconstructions of all the models we trained. [TODO: Describe the main observations, e.g. preservation of fine distal branches, connectivity of the tree, smoothness of the surface.]

Table 7: Reconstruction results of the DGCI model, reported as mean $\pm$ standard deviation. Red cells are still to be filled in.

Set	Continuous space			Discrete space		
	NM-V $\downarrow$	NM-E $\downarrow$	NM-F $\downarrow$	TD (%) $\uparrow$	BD (%) $\uparrow$	Dice (%) $\uparrow$
Validation	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]
Test	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]

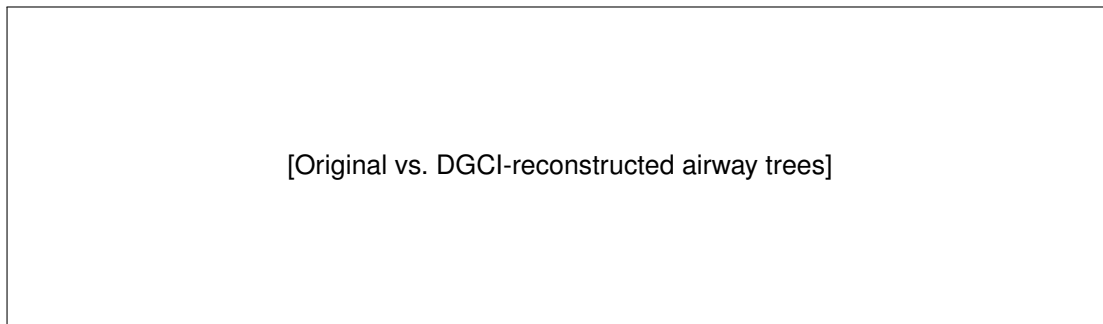


Figure 14: Ground-truth airway trees and their reconstructions by the DGCI model.

Anatomical Labeling of Airway Trees

To assign anatomical labels to our airway trees, we used the Implicit Point-Graph Network (IPGN) of Xie et al. (2025). We did not train this model ourselves: we used the weights released by the authors, trained on their Pulmonary Tree Labeling (PTL) dataset, and ran the model in inference mode only. This section describes how the model is formulated and how it was trained, so that its predictions on our data can be interpreted.

Task formulation

The input to the model is a binary volume containing a pulmonary tree (airway, artery or vein). The goal is a 19-class semantic segmentation of every foreground voxel: 18 classes correspond to segmental-level branches, i.e. the branches that supply the 18 pulmonary segments, and one class corresponds to the extra-pulmonary structures (Xie et al., 2025). Labels 1 to 10 are branches in the left lung, labels 11 to 18 are branches in the right lung, and label 19 is the extra-pulmonary part (for the airway, the trachea and main bronchi) (Xie et al., 2025).

Xie et al. point out that the hardest part of this task is labeling the branching points and the distal end points correctly. These key locations contain very few voxels compared to the rest of the tree, so a

model trained only on voxel-wise signals is dominated by easy voxels and can achieve a high voxel-level score while still mislabeling key locations (Xie et al., 2025). For this reason, IPGN also represents each tree as a skeleton graph whose nodes approximate these key points, and it is evaluated on both voxels and graph elements.

II Training data

PTL dataset. The model was trained on the PTL dataset, compiled by the authors from multiple medical centers (Xie et al., 2025). It contains 800 subjects, each with three volumes (airway, artery and vein) annotated with the 19 classes above. All volumes have size $N \times 512 \times 512$, with N between 181 and 798 slices and a slice spacing between 0.5 and 1.5 mm. The annotations were produced by a junior radiologist and verified by a senior radiologist. During training, each annotated volume is converted to a binary input, and the original labeled volume is the prediction target. The dataset was split randomly into 70% training, 10% validation and 20% test subjects (Xie et al., 2025).

Skeleton graphs. For each volume, a skeleton graph is built with the VesselVio software (Bumgarner and Nelson, 2022), which applies the thinning algorithm of Lee et al. (1994) to extract the centerlines of the tree. In the resulting graph, the centerline segments become edges representing anatomical connections, and the centerline intersections and distal end points become nodes representing the key points of the tree (Xie et al., 2025). No additional hyperparameters or modeling are involved: all spatial and connectivity information comes directly from the original volume.

Graph labels. Ground-truth labels for the graph are derived from the annotated volume in three steps, labeling edges before nodes because edge labels are deterministic while node labels can be ambiguous where classes meet (Xie et al., 2025):

1. **Graph simplification.** VesselVio also outputs, for each edge, the connected centerline points along it. Software inconsistencies can create cliques among these points; they are removed by keeping only the shortest path between the two nodes.
2. **Edge labeling.** The labels of the centerline points of each edge are read from the annotated volume, and the edge label is their majority vote.
3. **Node labeling.** A leaf node takes the label of its single edge, which is more reliable than reading it from the volume, since leaf coordinates can fall slightly outside the foreground. A node with several edges takes the majority vote of the labels of its connected edges.

II Model architecture

Overview. Instead of processing the dense volume with a CNN, IPGN works on two sparse representations of the tree: a point cloud, obtained by randomly sampling $M = 6,000$ foreground voxels, and the skeleton graph with N nodes (Xie et al., 2025). The point cloud is a downsampled version of the volume, while the graph describes the skeleton of the tree. The model has three stages (Figure 15): separate point and graph encoders, a sequence of Point-Graph Fusion layers that exchange information between the two branches, and an Implicit Point Module that turns the sparse predictions back into a dense labeled volume. The network without the implicit part is called the Point-Graph Network (PGN), and the full model is IPGN (Xie et al., 2025).

Notation. The point coordinates are $\mathbf{P} = \{\mathbf{p}_1, \dots, \mathbf{p}_M\} \in \mathbb{R}^{M \times 3}$ and the graph node coordinates are $\mathbf{G} = \{\mathbf{g}_1, \dots, \mathbf{g}_N\} \in \mathbb{R}^{N \times 3}$. A superscript (i) denotes the feature of an element after the i -th layer, e.g. $\mathbf{p}^{(i)}$ and $\mathbf{g}^{(i)}$, and $\frown$ denotes concatenation (Xie et al., 2025).

Point and graph encoders. The (x, y, z) coordinates of the points and of the graph nodes are the only input features. A point network and a graph network act as initial encoders and produce a 128-dimensional feature for each point and each node, $\mathbf{P}^{(0)} \in \mathbb{R}^{M \times 128}$ and $\mathbf{G}^{(0)} \in \mathbb{R}^{N \times 128}$ (Xie et al., 2025). The point encoder is either PointNet++ (Qi et al., 2017) or Point Transformer (Zhao et al., 2021), and the graph encoder is an 11-layer graph attention network (GAT) (Veličković et al., 2018) with 128 channels. The model we used has a **[TODO: PointNet++ / Point Transformer]** point encoder.

Point-Graph Fusion layers. The core of the model is a stack of l Point-Graph Fusion layers (Figure 16). Each layer takes $\mathbf{P}^{(i-1)}$ and $\mathbf{G}^{(i-1)}$ and outputs $\mathbf{P}^{(i)}$ and $\mathbf{G}^{(i)}$, using the 3D coordinates of the elements to find neighbors in the other branch (Xie et al., 2025).

Point to graph (ball query and grouping). Each graph node $\mathbf{g}$ collects all points within a ball of radius r around it, $\{\mathbf{p}_1, \dots, \mathbf{p}_b\}$, in the style of PointNet++ (Qi et al., 2017). An MLP $\mathcal{F}_1$ updates each of these point features independently, and feature-wise max pooling aggregates them into a point summary for the node (Xie et al., 2025):

$$\mathbf{g}_{bq}^{(i)} = \max_j \mathcal{F}_1(\mathbf{p}_j^{(i-1)}). \quad (37)$$

This summary is concatenated with the current node feature and passed through a shallow graph attention network $\mathcal{G}$ (Veličković et al., 2018), which performs graph convolution to produce the updated node feature:

$$\mathbf{g}^{(i)} = \mathcal{G}(\mathbf{g}^{(i-1)} \frown \mathbf{g}_{bq}^{(i)}), \quad \mathcal{G} : \mathbb{R}^{2D} \rightarrow \mathbb{R}^{D_{\text{next}}}. \quad (38)$$

Graph to point (feature propagation). Each point $\mathbf{p}$ finds its k nearest graph nodes $\{\mathbf{g}_1, \dots, \mathbf{g}_k\}$ in coordinate space, at distances $\{d_1, \dots, d_k\}$, and combines their features with weights given by the normalized inverse distances (Rossi et al., 2021; Xie et al., 2025):

$$\mathbf{p}_{fp}^{(i)} = \frac{\sum_{j=1}^k \mathbf{g}_j^{(i-1)} \cdot \frac{1}{d_j}}{\sum_{l=1}^k \frac{1}{d_l}}. \quad (39)$$

The result is concatenated with the incoming point feature, and an MLP $\mathcal{F}_2$ projects it to the next layer's dimension:

$$\mathbf{p}^{(i)} = \mathcal{F}_2(\mathbf{p}^{(i-1)} \frown \mathbf{p}_{fp}^{(i)}), \quad \mathcal{F}_2 : \mathbb{R}^{2D} \rightarrow \mathbb{R}^{D_{\text{next}}}. \quad (40)$$

In this way, the points receive the structural context of the skeleton, and the graph receives the local shape information of the surrounding points. After the last fusion layer, a lightweight MLP and a GNN project the point and node features to 19-dimensional class scores (Xie et al., 2025).

Implicit Point Module. PGN only labels the 6,000 sampled points and the graph nodes. Labeling every foreground voxel by repeatedly running PGN on new point clouds would be inefficient, because the graph and the global shape of the point cloud do not change between runs (Xie et al., 2025). The Implicit Point Module instead treats the point features learned by PGN as a continuous feature field that can be queried at any location (Figure 15). For a query point $\mathbf{p}_q = (x_q, y_q, z_q)$, which does not need to belong to the sampled point cloud:

1. its k nearest neighbors $\{\mathbf{p}_1, \dots, \mathbf{p}_k\}$ in the sampled point cloud are found;

2. for each neighbor, its features from all stages of PGN are concatenated into a multi-stage feature $\mathbf{z}_k = \mathbf{p}_k^{(0)} \frown \mathbf{p}_k^{(1)} \frown \dots \frown \mathbf{p}_k^{(l)}$, and these are combined into a feature $\mathbf{z}_q$ for the query point by inverse-distance feature propagation, as in Equation (39);
3. an MLP $\mathcal{H}$ maps $\mathbf{z}_q$ to a 19-dimensional vector of class scores (Xie et al., 2025).

Using features from all stages, rather than only the last fusion layer, gave the best results in the authors' ablation study (Xie et al., 2025).

Hyperparameters. The ball query uses a radius $r = 0.1$ and at most 24 points per node; $\mathcal{G}$ is a 3-layer GAT; the k -nearest-neighbor search in both the graph-to-point fusion and the Implicit Point Module uses $k = 3$; and all intermediate fusion features are 128-dimensional (Xie et al., 2025). With a PointNet++ encoder, PGN has about 3.6M parameters, and the Implicit Point Module adds about 0.1M (Xie et al., 2025).

[IPGN overview figure]

Figure 15: Overview of the Implicit Point-Graph Network (IPGN), adapted from Xie et al. (2025). The binary volume is converted into a skeleton graph and a point cloud, Point-Graph Fusion layers exchange features between the two branches, and the Implicit Point Module produces the dense labeled volume.

[Point-Graph Fusion layer figure]

Figure 16: A Point-Graph Fusion layer, adapted from Xie et al. (2025). Graph nodes gather point features by ball query and grouping; points gather graph features by k -nearest-neighbor feature propagation.

II Training

The released model was trained by Xie et al. in two stages (Xie et al., 2025).

Encoder pre-training. First, the point encoder and the graph encoder were trained independently on the point and graph versions of the 19-class labeling task, and then frozen. The point encoders were

trained for 120 epochs with a learning rate of 0.002, and the GAT graph encoder for 240 epochs with a learning rate of 0.02 (Xie et al., 2025).

Joint training. Second, PGN and the Implicit Point Module were trained together. In each step, PGN is run on $M = 6,000$ sampled points and the N graph nodes, producing multi-stage features and predictions for all of them. A second, independent set of $M' = 6,000$ foreground points is then sampled and passed through the Implicit Point Module as query points. The cross-entropy loss is applied to all $M + M'$ point predictions and all N graph predictions (Xie et al., 2025). This stage ran for 100 epochs with a learning rate of 0.01, halved every 45 epochs. Random rotation, shifting and scaling were used as data augmentation to avoid overfitting (Xie et al., 2025).

II Inference

Inference follows the procedure of Xie et al. (2025):

1. **Pre-processing.** The binary airway volume is skeletonized to obtain the skeleton graph (Section “Training data”), and 6,000 foreground voxels are randomly sampled as the point cloud.
2. **Point-Graph Network.** A single forward pass of PGN on the graph and point cloud gives the multi-stage point features and the labels of the graph nodes.
3. **Dense labeling.** All remaining foreground voxels are grouped into batches and passed through the Implicit Point Module as query points, which gives a label for every voxel of the tree.

Because PGN only runs once, dense labeling is fast: the authors report around 1.3 s per airway tree with a PointNet++ encoder and 2.3 s with Point Transformer, against 5.7 s and 12.1 s when PGN is run repeatedly over the whole volume (Xie et al., 2025).

II Reported performance

Xie et al. evaluate the model with micro-averaged Dice scores, chosen because of the class imbalance, at two levels (Xie et al., 2025). At the *voxel level*, the Dice score measures the overlap between the predicted and ground-truth labeled volumes and reflects overall performance. At the *graph level*, the Dice score is computed on the labels of the skeleton nodes and edges and reflects performance at the key locations of the tree.

Table 8 summarizes the results reported for the airway tree on the PTL test set. IPGN outperforms CNN, point-based and graph-based baselines at both levels, and its dense predictions from the Implicit Point Module are almost identical to those of the full PGN (Xie et al., 2025).

II Application to our airway trees

We applied the released IPGN model with the [TODO: PointNet++ / Point Transformer] encoder to our [TODO: number] airway trees. Each tree was [TODO: binarized and resampled to ...], skeletonized with VesselVio, and labeled following the inference procedure described in Section “Inference”. Examples of the labeled trees are shown in Figure 17. [TODO: Since our data has no ground-truth segmental labels, the predictions were assessed visually / against ...] The resulting labels are

Table 8: Airway labeling results on the PTL test set as reported by Xie et al. (2025): micro-averaged Dice at voxel and graph (node, edge) level, and dense inference time per tree.

Method	Voxel (%)	Node (%)	Edge (%)	Time (s)
3D U-Net (down-sampled)	58.5	61.3	62.3	8.16
Point Transformer	87.8	87.2	87.4	10.97
GAT	86.3	92.7	89.1	1.15
IPGN (PointNet++)	87.5	95.1	92.3	1.30
IPGN (Point Transformer)	88.5	95.0	92.7	2.32

used in Section “Morphological Metrics” to compute the morphological descriptors of each anatomical class.

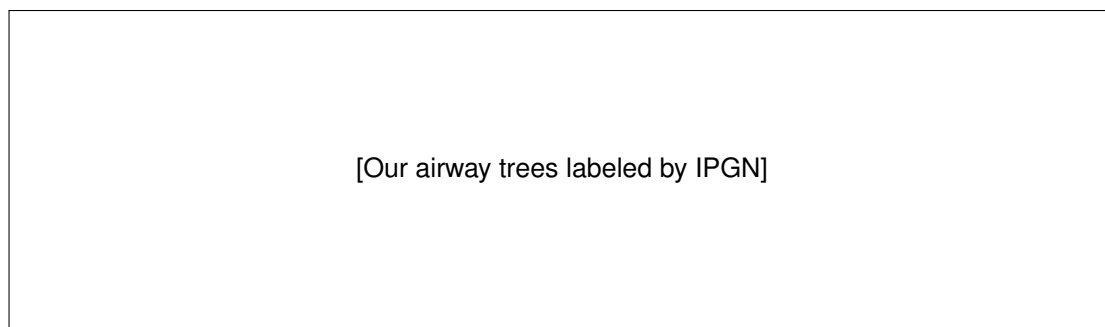


Figure 17: Anatomical labels predicted by IPGN on examples of our airway trees. Colours denote the 19 classes.

I Morphological Analysis of Airway Trees

Once every airway tree is anatomically labeled (Section “Labelling”), we describe its shape with the six morphological descriptors of AirwaySignature, introduced with the AirMorph framework by Zhang et al. (2025): Stenosis (S), Ectasia (E), Tortuosity (T), Length (L), Divergence (D) and Complexity (C). The descriptors capture local variations in airway shape that cannot be seen from the binary segmentation alone, and they are defined per anatomical class, so that each value refers to a specific bronchus (a segmental or a lobar branch) (Zhang et al., 2025). For each tree, the result is a compact, anatomically aligned signature: one value of each descriptor for every anatomical class present in the tree.

II Overview

Zhang et al. (2025) divide the six descriptors into two groups (Figure 18):

- **Branch-level descriptors** (S , E , T) are first computed on each individual branch of the airway skeleton and then averaged over all branches that belong to the same anatomical class. Stenosis measures the strongest narrowing of a branch, Ectasia its strongest dilation, and Tortuosity how much its 3D trajectory bends.
- **Subtree-level descriptors** (L , D , C) are computed on the whole subtree of an anatomical class, i.e. on all connected branches that share the same anatomical context. Length measures the geodesic length of the subtree, Divergence the angular spread of its terminal branches, and Complexity the irregularity and density of its branching through a fractal dimension.

The anatomical class K over which the descriptors are grouped can be a segmental or a lobar bronchus (Zhang et al., 2025). In our case the labels come from IPGN, which provides 18 segmental classes; **[TODO: lobar classes were obtained by merging the segmental labels of each lobe / we computed the descriptors at segmental level only].**

Table 9 summarizes the six descriptors.

Table 9: The six AirwaySignature descriptors of Zhang et al. (2025).

Descriptor	Computed on	Range	Higher value means
Stenosis (S)	Each branch, then averaged	$[0\%, 100\%]$	stronger narrowing
Ectasia (E)	Each branch, then averaged	$[100\%, +\infty)$	stronger dilation
Tortuosity (T)	Each branch, then averaged	$[0, 1]$	stronger bending
Length (L)	Class subtree	$[0, +\infty)$ mm	longer subtree
Divergence (D)	Class subtree	$[0, 1]$	wider angular spread
Complexity (C)	Class subtree	$[0, +\infty)$	more complex branching

[Definition of the six morphological descriptors]

Figure 18: The six morphological descriptors of AirwaySignature, adapted from Zhang et al. (2025).

Common pre-processing

Stenosis and Ectasia are both based on the local airway radius along the centerline, which is obtained as follows (Zhang et al., 2025):

1. **Radius field.** A 3D Euclidean distance transform (EDT) is applied to the binary airway mask V_{airway} , taking the voxel spacing s into account. At every foreground voxel it gives the distance to the nearest background voxel, i.e. an approximation of the local airway radius in mm.
2. **Centerline radii.** The distance field is masked with the airway skeleton $\mathcal{S}$, keeping only the values on centerline voxels. This gives the radius field along the skeleton:

$$R = \text{EDT}(V_{\text{airway}}; s) \cdot \mathcal{S}. \quad (41)$$

3. **Skeleton parsing.** The skeleton is split into individual branches using a 26-connected neighborhood and removing the junction voxels, so that each branch is spatially isolated. Each parsed branch b receives the anatomical label of the region it lies in.

For every parsed branch b , the minimum, mean and maximum radius along its centerline are then computed:

$$r_{\min}^{(b)} = \min_{x \in b} R(x), \quad r_{\text{mean}}^{(b)} = \frac{1}{|b|} \sum_{x \in b} R(x), \quad r_{\max}^{(b)} = \max_{x \in b} R(x). \quad (42)$$

II Branch-level descriptors

Stenosis (S). Stenosis quantifies airway narrowing. Zhang et al. (2025) define it as the reduction of the radius at the point of strongest constriction, R_{narrow} , relative to a reference radius R_{ref} , the mean radius of a healthy proximal airway segment:

$$\text{Stenosis} = \left(1 - \frac{R_{\text{narrow}}}{R_{\text{ref}}}\right) \times 100\%. \quad (43)$$

In their implementation, both quantities are taken from the radius statistics of each parsed branch: R_{narrow} is the branch's minimum radius and R_{ref} its mean radius, which gives the local stenosis ratio of branch b :

$$S_b = 1 - \frac{r_{\min}^{(b)}}{r_{\text{mean}}^{(b)}}. \quad (44)$$

The stenosis of an anatomical class K is the average over all branches carrying that label:

$$S_K = \frac{1}{|B_K|} \sum_{b \in B_K} S_b, \quad (45)$$

where B_K is the set of parsed branches with label K . If a class is absent from the tree, its score is set to -1 (Zhang et al., 2025). Stenosis ranges from 0% to 100%, and higher values mean a more severe narrowing.

Ectasia (E). Ectasia complements Stenosis: instead of narrowing, it captures over-expansion of the airway lumen (Zhang et al., 2025). It is defined as the ratio between the maximum radius and the reference radius, which in the implementation is again the mean radius of each parsed branch:

$$\text{Ectasia} = \frac{R_{\text{max}}}{R_{\text{ref}}} \times 100\%, \quad E_b = \frac{r_{\text{max}}^{(b)}}{r_{\text{mean}}^{(b)}}. \quad (46)$$

As for Stenosis, the class value E_K is the average of E_b over all branches with label K , and absent classes receive -1 . Ectasia ranges from 100% upwards, and higher values mean a stronger dilation (Zhang et al., 2025).

Tortuosity (T). Tortuosity describes the local curvature of a branch from its volumetric geometry (Zhang et al., 2025). For a branch region Ω_l (the set of its voxels):

1. the voxel coordinates are converted to physical space using the voxel spacing;
2. principal component analysis (PCA) is applied to these coordinates, and the two endpoints $\mathbf{S}$ and $\mathbf{E}$ of the branch are taken along the first principal axis, i.e. the main direction of the branch;
3. the voxel $\mathbf{P}$ that lies furthest from the segment $\overline{SE}$ is found:

$$\mathbf{P} = \arg \max_{\mathbf{x}_i \in \Omega_l} \text{dist}(\mathbf{x}_i, \overline{SE}); \quad (47)$$

4. the tortuosity is the angle formed at $\mathbf{P}$ by the vectors pointing to the two endpoints:

$$T = \alpha = \angle(\overrightarrow{PS}, \overrightarrow{PE}) = \arccos \frac{\overrightarrow{PS} \cdot \overrightarrow{PE}}{\|\overrightarrow{PS}\| \|\overrightarrow{PE}\|}. \quad (48)$$

This angle reflects how much the branch bends or deflects. It lies in $[0, \pi]$ and is then normalized to $[0, 1]$, with values close to 1 indicating a high degree of tortuosity (Zhang et al., 2025). **[TODO: State the normalization you used, e.g. $T = 1 - \alpha/\pi$.]** As for Stenosis and Ectasia, the class value T_K is the average over all branches with label K .

II Subtree-level descriptors

The remaining three descriptors work on the centerline tree, in which each node carries an anatomical label, a generation number (its depth in the tree) and a length. For each class K , they first identify all terminal (leaf) nodes labeled K , denoted T_K , and their lowest common ancestor (LCA), found by a generation-aware traversal of the tree (Zhang et al., 2025). The LCA is the node where the subtree of class K begins.

Length (L). Length measures the geometric elongation of a segment or lobe as a geodesic length, accumulated branch by branch along the skeleton (Zhang et al., 2025). If the LCA itself belongs to class K , the length is the average, over all leaves of class K , of the path length from the LCA to that leaf:

$$L_K = \frac{1}{|T_K|} \sum_{v \in T_K} \sum_{n \in \text{Path}(\text{LCA}, v)} \ell(n), \quad (49)$$

where $\text{Path}(\text{LCA}, v)$ is the set of nodes on the path from the LCA to leaf v and $\ell(n)$ is the length of node n . If the LCA does not belong to class K (for example, when it is the trachea), each path is re-rooted at the nearest downstream node labeled K , so that only the part of the tree belonging to class K is measured (Zhang et al., 2025). Length is expressed in mm and ranges from 0 upwards.

Divergence (D). Divergence measures the spatial dispersion of the branches of a class, i.e. how widely they spread from their common origin, using the smallest cone that encloses them (Zhang et al., 2025). The LCA of the class- K leaves is taken as the apex of the cone, and $\mathbf{v}_i$ is the normalized direction vector from the apex to leaf i . The axis of the cone is the unit vector $\mathbf{u}^*$ that maximizes the smallest cosine similarity with all leaf directions:

$$\mathbf{u}^* = \arg \max_{\|\mathbf{u}\|=1} \min_i \langle \mathbf{u}, \mathbf{v}_i \rangle. \quad (50)$$

The divergence angle is twice the largest angular deviation of a leaf direction from this axis, i.e. the full opening angle of the enclosing cone:

$$\theta = 2 \cdot \arccos \left(\min_i \langle \mathbf{u}^*, \mathbf{v}_i \rangle \right). \quad (51)$$

The angle lies in $[0, \pi]$ and is normalized to $[0, 1]$. A smaller angle means that the branches of the class are less separated and less angularly dispersed, which can be affected by pulmonary disease (Zhang et al., 2025).

Complexity (C). Complexity measures the geometric complexity of the branches of a class with a box-counting fractal dimension, which captures the spatial irregularity and branching density of the skeleton (Zhang et al., 2025). For each class K :

1. the skeleton voxels labeled K are extracted as a binary mask, $\mathcal{S}_K = (\mathcal{S} > 0) \wedge (Y = K)$;

2. this mask is cropped to its bounding box and zero-padded to a fixed 3D size;
3. the volume is covered with cubic boxes of sizes $s \in \{2^1, 2^2, \dots, 2^k\}$, and for each size the number $N(s)$ of boxes containing at least one skeleton voxel is counted.

The complexity is the box-counting dimension

$$C = \lim_{s \rightarrow 0} \frac{\log N(s)}{\log(1/s)}, \quad (52)$$

estimated in practice as the slope of a linear regression of $\log N(s)$ against $\log(1/s)$ (Zhang et al., 2025). A straight line has a dimension close to 1, while a densely branching structure that fills more of the space has a higher value. Complexity ranges from 0 upwards, and larger values denote a more complex branching pattern.

II Application to our airway trees

We computed the six descriptors for all [TODO: number] labeled airway trees, at [TODO: segmental / lobar] level, following the procedures above. [TODO: Describe any implementation choices, e.g. the skeletonization method, the normalization of T and D , the node lengths $\ell(n)$, and the maximum box size 2^k .] Examples of the resulting signatures are shown in Figure 19. These descriptors are used in [TODO: Section “Clustering”?] to [TODO: describe their use, e.g. to characterize the clusters].

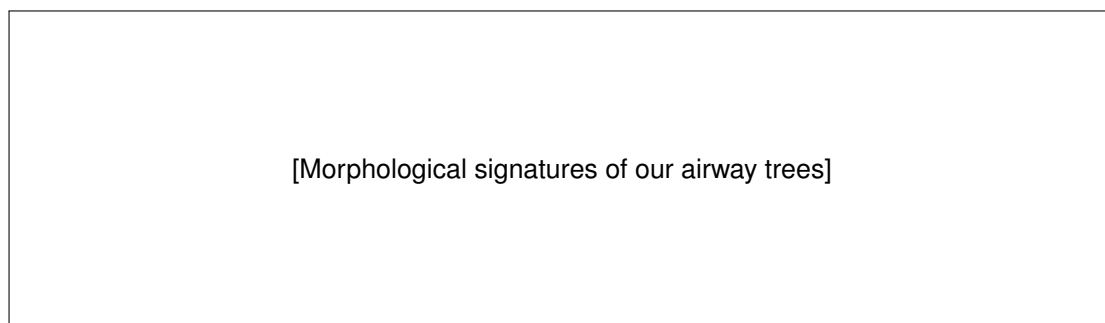


Figure 19: Morphological descriptors computed on examples of our labeled airway trees.

I Clustering

[Describe the clustering method, its inputs and parameters (e.g. number of clusters), how the result was evaluated, and what the clusters show.]

I Conclusions and Recommendations

II Conclusions

[Summarise how the objectives stated in the introduction have been met and make some concluding remarks.]

II Recommendations

[Briefly describe the proposed plans of action for the future.]



I Appendices

II Appendix A: Title of the Document

[Any document or information needed to support an argument or clarify something mentioned in the body of the report. Number and title every table and figure.]



List of References

- Abbasi, Z. and Boozarjomehry, R. B. (2021). Modeling of human conducting airways by stochastic parametric L-system. *The European Physical Journal Plus*, 136:197.
- Agujetas, R., Barrio-Perotti, R., Ferrera, C., Pandal-Blanco, A., Walters, D. K., and Fernández-Tena, A. (2020). Construction of a hybrid lung model by combining a real geometry of the upper airways and an idealized geometry of the lower airways. *Computer Methods and Programs in Biomedicine*, 196:105613.
- Avants, B. B., Epstein, C. L., Grossman, M., and Gee, J. C. (2008). Symmetric diffeomorphic image registration with cross-correlation: Evaluating automated labeling of elderly and neurodegenerative brain. *Medical Image Analysis*, 12(1):26–41.
- Avants, B. B., Yushkevich, P., Pluta, J., Minkoff, D., Korczykowski, M., Detre, J., and Gee, J. C. (2010). The optimal template effect in hippocampus studies of diseased populations. *NeuroImage*, 49(3):2457–2466.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- Batten, J., Schaap, M., Sinclair, M., Bai, Y., and Glocker, B. (2025). Vector representations of vessel trees. *arXiv preprint arXiv:2506.11163*.
- Belchi, F., Pirashvili, M., Conway, J., Bennett, M., Djukanovic, R., and Brodzki, J. (2018). Lung topology characteristics in patients with chronic obstructive pulmonary disease. *Scientific Reports*, 8(1):5341.
- Bodduluri, S. et al. (2018). Airway fractal dimension predicts respiratory morbidity and mortality in COPD. *Journal of Clinical Investigation*, 128(12):5374–5382.
- Bodduluri, S., Puliakote, A. K., Nakhmani, A., Charbonnier, J.-P., Reinhardt, J. M., and Bhatt, S. P. (2021). Computed tomography-based airway surface area-to-volume ratio for phenotyping airway remodeling in chronic obstructive pulmonary disease. *American Journal of Respiratory and Critical Care Medicine*, 203(2):185–191.
- Bordas, R., Lefevre, C., Veeckmans, B., Pitt-Francis, J., Fetita, C., Brightling, C. E., et al. (2015). Development and analysis of patient-based complete conducting airways models. *PLoS ONE*, 10(12):e0144105.
- Boyden, E. A. (1949). A synthesis of the prevailing patterns of the bronchopulmonary segments in the light of their variations. *CHEST Journal*, 15(6):657–668.
- Brock, A., Lim, T., Ritchie, J. M., and Weston, N. (2016). Generative and discriminative voxel modeling with convolutional neural networks. *arXiv preprint arXiv:1608.04236*.
- Bülow, T., Lorenz, C., Wiemker, R., and Honko, J. (2006). Point based methods for automatic bronchial tree matching and labeling. In *SPIE Medical Imaging*, volume 6143, pages 225–234.
- Bumgarner, J. R. and Nelson, R. J. (2022). Open-source analysis and visualization of segmented vasculature datasets with VesselVio. *Cell Reports Methods*, 2(4):100189.
- Clevert, D.-A., Unterthiner, T., and Hochreiter, S. (2016). Fast and accurate deep network learning by exponential linear units (ELUs). In *International Conference on Learning Representations (ICLR)*.

- Davoodi, A. and Boozarjomehry, R. B. (2016). Developmental model of an automatic production of the human bronchial tree based on L-system. *Computer Methods and Programs in Biomedicine*, 132:1–10.
- Feldman, P., Fainstein, M., Siless, V., Delrieux, C., and Iarussi, E. (2023). VesselVAE: Recursive variational autoencoders for 3D blood vessel synthesis. In *Medical Image Computing and Computer Assisted Intervention (MICCAI)*.
- Feldman, P., Sinnona, M., Delrieux, C., Siless, V., and Iarussi, E. (2025). VesselGPT: Autoregressive modeling of vascular geometry. *arXiv preprint arXiv:2505.13318*.
- Feragen, A., Hauberg, S., Nielsen, M., and Lauze, F. (2011a). Means in spaces of tree-like shapes. In *IEEE International Conference on Computer Vision (ICCV)*.
- Feragen, A., Lo, P., Gorbunova, V., Nielsen, M., Dirksen, A., Reinhardt, J. M., Lauze, F., and de Bruijne, M. (2011b). An airway tree-shape model for geodesic airway branch labeling. In *MICCAI Workshop on Mathematical Foundations of Computational Anatomy (MFCA)*, pages 123–134.
- Feragen, A., Petersen, J., Grimm, D., Dirksen, A., Pedersen, J. H., Borgwardt, K., and de Bruijne, M. (2013). Geometric tree kernels: Classification of COPD from airway tree geometry. In *Information Processing in Medical Imaging (IPMI)*, volume 7917 of *Lecture Notes in Computer Science*, pages 171–183.
- Feragen, A., Petersen, J., Owen, M., Lo, P., Thomsen, L. H., Wille, M. M. W., Dirksen, A., and de Bruijne, M. (2012). A hierarchical scheme for geodesic anatomical labeling of airway trees. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, volume 7512 of *Lecture Notes in Computer Science*, pages 147–155.
- Florens, M., Sapoval, B., and Filoche, M. (2010). An anatomical and functional model of the human tracheobronchial tree. *Journal of Applied Physiology*, 110(3):756–763.
- Garcia-Uceda, A., Selvan, R., Saghir, Z., Tiddens, H. A. W. M., and de Bruijne, M. (2021). Automatic airway segmentation from computed tomography using robust and efficient 3-D convolutional neural networks. *Scientific Reports*, 11:16001.
- Geronzi, L., Fanni, B. M., De Jong, B., Roest, G., Kenjeres, S., Celi, S., and Biancolini, M. E. (2024). A parametric 3D model of human airways for particle drug delivery and deposition. *Fluids*, 9:27.
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*.
- Gropp, A., Yariv, L., Haim, N., Atzmon, M., and Lipman, Y. (2020). Implicit geometric regularization for learning shapes. In *International Conference on Machine Learning (ICML)*.
- Ha, D., Dai, A. M., and Le, Q. V. (2017). HyperNetworks. In *International Conference on Learning Representations (ICLR)*.
- Hasegawa, M., Nasuhara, Y., Onodera, Y., Makita, H., Nagai, K., Fuke, S., Ito, Y., Betsuyaku, T., and Nishimura, M. (2006). Airflow limitation and airway dimensions in chronic obstructive pulmonary disease. *American Journal of Respiratory and Critical Care Medicine*, 173(12):1309–1315.
- Hegedűs, C. J., Balásházy, I., and Farkas, Á. (2004). Detailed mathematical description of the geometry of airway bifurcations. *Respiratory Physiology & Neurobiology*, 141(1):99–114.

- Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., and Lerchner, A. (2017). beta-VAE: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations (ICLR)*.
- Ho, T. T., Tran, M. T., Cui, X., et al. (2024). Human-airway surface mesh smoothing based on graph convolutional neural networks. *Computer Methods and Programs in Biomedicine*, 246:108061.
- Horsfield, K. (1986). Morphometry of airways. In *Handbook of Physiology, The Respiratory System III*, pages 75–87. American Physiological Society.
- Horsfield, K., Dart, G., Olson, D. E., Filley, G. F., and Cumming, G. (1971). Models of the human bronchial tree. *Journal of Applied Physiology*, 31(2):207–217.
- Ilegbusi, O. J., Islam, A., and Santhanam, A. P. (2023). Computational modelling of airflow in distal airways using hybrid lung model. *Mathematical and Computer Modelling of Dynamical Systems*, 29:186–207.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*.
- Kaftan, J. N., Kiraly, A. P., Naidich, D. P., and Novak, C. L. (2006). A novel multi-purpose tree and path matching algorithm with application to airway trees. In *SPIE Medical Imaging*, volume 6143, pages 215–224.
- Kaji, S., Tanabe, N., Maetani, T., Shiraishi, Y., Sakamoto, R., Oguma, T., Suzuki, K., Terada, K., Fukui, M., Muro, S., Sato, S., and Hirai, T. (2024). Quantification of airway structures by persistent homology. *IEEE Transactions on Medical Imaging*, 43(8):2758–2768.
- Kamiya, A., Togawa, T., and Yamamoto, A. (1974). Theoretical relationship between the optimal models of the vascular tree. *Bulletin of Mathematical Biology*, 36(3):311–323.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.
- Kingma, D. P. and Welling, M. (2014). Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*.
- Kirby, M. et al. (2018). Total airway count on computed tomography and the risk of chronic obstructive pulmonary disease progression. Findings from a population-based study. *American Journal of Respiratory and Critical Care Medicine*, 197(1):56–65.
- Kitaoka, H., Park, Y., Tschirren, J., Reinhardt, J. M., Sonka, M., McLennan, G., and Hoffman, E. A. (2002). Automated nomenclature labeling of the bronchial tree in 3D-CT lung images. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 1–11.
- Kitaoka, H., Takaki, R., and Suki, B. (1999). A three-dimensional model of the human airway tree. *Journal of Applied Physiology*, 87(6):2207–2217.
- Lauria, M., Singhrao, K., Stiehl, B., et al. (2022). Automatic triangulated mesh generation of pulmonary airways from segmented lung 3DCTs for computational fluid dynamics. *International Journal of Computer Assisted Radiology and Surgery*, 17:185–197.
- Lee, T.-C., Kashyap, R. L., and Chu, C.-N. (1994). Building skeleton models via 3-D medial surface axis thinning algorithms. *CVGIP: Graphical Models and Image Processing*, 56(6):462–478.

- Lin, C.-L., Tawhai, M. H., and Hoffman, E. A. (2013). Multiscale image-based modeling and simulation of gas flow and particle transport in the human lungs. *Wiley Interdisciplinary Reviews: Systems Biology and Medicine*, 5:643–655.
- Lin, C.-L., Tawhai, M. H., McLennan, G., and Hoffman, E. A. (2009). Multiscale simulation of gas flow in subject-specific models of the human lung. *IEEE Engineering in Medicine and Biology Magazine*, 28(3):25–33.
- Lorensen, W. E. and Cline, H. E. (1987). Marching cubes: A high resolution 3D surface construction algorithm. *ACM SIGGRAPH Computer Graphics*, 21(4):163–169.
- Maetani, T. et al. (2023). Physiological impacts of computed tomography airway dysanapsis, fractal dimension, and branch count in asymptomatic never smokers. *Journal of Applied Physiology*, 134(1):20–27.
- Marchandise, E., Geuzaine, C., and Remacle, J.-F. (2013). Cardiovascular and lung mesh generation based on centerlines. *International Journal for Numerical Methods in Biomedical Engineering*, 29(6):655–682.
- Mattes, D., Haynor, D. R., Vesselle, H., Lewellen, T. K., and Eubank, W. (2003). PET-CT image registration in the chest using free-form deformations. *IEEE Transactions on Medical Imaging*, 22(1):120–128.
- Metzen, J. H., Kröger, T., Schenk, A., Zidowitz, S., Peitgen, H.-O., and Jiang, X. (2009). Matching of anatomical tree structures for registration of medical images. *Image and Vision Computing*, 27:923–933.
- Miyawaki, S., Tawhai, M. H., Hoffman, E. A., Wenzel, S. E., and Lin, C.-L. (2017). Automatic construction of subject-specific human airway geometry including trifurcations based on a CT-segmented airway skeleton and surface. *Biomechanics and Modeling in Mechanobiology*, 16(2):583–596.
- Montesantos, S., Katz, I., Fleming, J., Majoral, C., Pichelin, M., Dubau, C., et al. (2013). Airway morphology from high resolution computed tomography in healthy subjects and patients with moderate persistent asthma. *The Anatomical Record*, 296(6):852–866.
- Montesantos, S., Katz, I., Pichelin, M., and Caillibotte, G. (2016). The creation and statistical evaluation of a deterministic model of the human bronchial tree from HRCT images. *PLoS ONE*, 11(12):e0168026.
- Museth, K. (2013). VDB: High-resolution sparse volumes with dynamic topology. *ACM Transactions on Graphics*, 32(3):27:1–27:22.
- Nakamura, M., Wada, S., Miki, T., Shimada, Y., Suda, Y., and Tamura, G. (2008). Automated segmentation and morphometric analysis of the human airway tree from multidetector CT images. *The Journal of Physiological Sciences*, 58:493–498.
- Nakano, Y., Muro, S., Sakai, H., Hirai, T., Chin, K., Tsukino, M., Nishimura, K., Itoh, H., Paré, P. D., Hogg, J. C., and Mishima, M. (2000). Computed tomographic measurements of airway dimensions and emphysema in smokers. Correlation with lung function. *American Journal of Respiratory and Critical Care Medicine*, 162(3):1102–1108.
- Nan, Y., Xing, X., Wang, S., Tang, Z., et al. (2024). Hunting imaging biomarkers in pulmonary fibrosis: Benchmarks of the AIB23 challenge. *Medical Image Analysis*, 97:103253.

- Nousias, S., Zacharaki, E. I., and Moustakas, K. (2020). AVATREE: An open-source computational modelling framework modelling anatomically valid airway tree conformations. *PLOS ONE*, 15(4):e0230259.
- Odena, A., Dumoulin, V., and Olah, C. (2016). Deconvolution and checkerboard artifacts. *Distill*.
- Oguma, T. et al. (2015). Longitudinal shape irregularity of airway lumen assessed by CT in patients with bronchial asthma and COPD. *Thorax*, 70(8):719–724.
- Ortiz-Puerta, D., Cox, A., and Hurtado, D. E. (2022). Snakes isogeometric analysis (SIGA): Towards accurate and flexible geometrical models of the respiratory airways. *Computer Methods in Applied Mechanics and Engineering*, 394:114841.
- Palágyi, K., Tschirren, J., Hoffman, E. A., and Sonka, M. (2006). Quantitative analysis of pulmonary airway tree structures. *Computers in Biology and Medicine*, 36(9):974–996.
- Park, J. J., Florence, P., Straub, J., Newcombe, R., and Lovegrove, S. (2019). DeepSDF: Learning continuous signed distance functions for shape representation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Patel, B. D., Coxson, H. O., Pillai, S. G., et al. (2008). Airway wall thickening and emphysema show independent familial aggregation in chronic obstructive pulmonary disease. *American Journal of Respiratory and Critical Care Medicine*, 178(5):500–505.
- Pennati, F., Aliboni, L., and Aliverti, A. (2024). Modeling realistic geometries in human intrathoracic airways. *Diagnostics*, 14(17):1979.
- Prabhakar, C., Shit, S., Amiranashvili, T., Li, H. B., and Menze, B. (2025). Semantically consistent discrete diffusion for 3D biological graph modeling. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 594–604. Springer.
- Prabhakar, C., Shit, S., Musio, F., Yang, K., Amiranashvili, T., Paetzold, J. C., Li, H. B., and Menze, B. (2024). 3D vessel graph generation using denoising diffusion. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 3–13. Springer.
- Prabhakar, C., Wittmann, B., Büschl, P., Li, H. B., Menze, B., and Shit, S. (2026). Sparse representation learning for vessels. *arXiv preprint arXiv:2605.01382*.
- Qi, C. R., Yi, L., Su, H., and Guibas, L. J. (2017). PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Ren, X., Huang, J., Zeng, X., Museth, K., Fidler, S., and Williams, F. (2024). XCube: Large-scale 3D generative modeling using sparse voxel hierarchies. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Rossi, E., Kenlay, H., Gorinova, M. I., Chamberlain, B. P., Dong, X., and Bronstein, M. M. (2021). On the unreasonable effectiveness of feature propagation in learning on graphs with missing node features. *arXiv preprint arXiv:2111.12128*.
- Shit, S., Paetzold, J. C., Sekuboyina, A., Ezhov, I., Unger, A., Zhylka, A., Pluim, J. P. W., Bauer, U., and Menze, B. H. (2021). clDice – a novel topology-preserving loss function for tubular structure segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Sitzmann, V., Martel, J., Bergman, A., Lindell, D., and Wetzstein, G. (2020). Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- Smith, B. M. et al. (2020). Association of dysanapsis with chronic obstructive pulmonary disease among older adults. *JAMA*, 323(22):2268.
- Sørensen, L., Lo, P., Dirksen, A., Petersen, J., and de Bruijne, M. (2011). Dissimilarity-based classification of anatomical tree structures. In *Information Processing in Medical Imaging (IPMI)*, Lecture Notes in Computer Science, pages 475–485.
- Sutskever, I., Martens, J., Dahl, G., and Hinton, G. (2013). On the importance of initialization and momentum in deep learning. In *International Conference on Machine Learning (ICML)*.
- Tawhai, M., Clark, A., Donovan, G., and Burrowes, K. (2011). Computational modeling of airway and pulmonary vascular structure and function: Development of a “Lung Physiome”. *Critical Reviews in Biomedical Engineering*, 39(4).
- Tawhai, M. H., Hoffman, E. A., and Lin, C.-L. (2009). The lung physiome: merging imaging-based measures with predictive computational models. *Wiley Interdisciplinary Reviews: Systems Biology and Medicine*, 1(1):61–72.
- Tawhai, M. H., Hunter, P., Tschirren, J., Reinhardt, J., McLennan, G., and Hoffman, E. A. (2004). CT-based geometry analysis and finite element models of the human and ovine bronchial tree. *Journal of Applied Physiology*, 97(6):2310–2321.
- Tawhai, M. H., Pullan, A. J., and Hunter, P. J. (2000). Generation of an anatomically based three-dimensional model of the conducting airways. *Annals of Biomedical Engineering*, 28(7):793–802.
- Thévenaz, P., Ruttimann, U. E., and Unser, M. (1998). A pyramid approach to subpixel registration based on intensity. *IEEE Transactions on Image Processing*, 7(1):27–41.
- Tschirren, J., McLennan, G., Palágyi, K., Hoffman, E. A., and Sonka, M. (2005). Matching and anatomical labeling of human airway tree. *IEEE Transactions on Medical Imaging*, 24(12):1540–1547.
- Tustison, N. J., Cook, P. A., Holbrook, A. J., Johnson, H. J., Muschelli, J., Devenyi, G. A., et al. (2021). The ANTsX ecosystem for quantitative biological and medical imaging. *Scientific Reports*, 11:9068.
- Ulyanov, D., Vedaldi, A., and Lempitsky, V. (2016). Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*.
- van Ginneken, B., Baggerman, W., and van Rikxoort, E. (2008). Robust segmentation and anatomical labeling of the airway tree from thoracic CT scans. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 219–226.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations (ICLR)*.
- Wang, C., Bassingthwaighe, J., and Weissman, L. (1992). Bifurcating distributive system using Monte Carlo method. *Mathematical and Computer Modelling*, 16(3):91–98.
- Weng, Z., Yang, J., Xie, K., Zhou, B., and Cai, W. (2026). Learning topology-aware implicit field for unified pulmonary tree modeling with incomplete topological supervision. *arXiv preprint arXiv:2602.02186*.
- Xie, K., Yang, J., Wei, D., Weng, Z., and Fua, P. (2025). Efficient anatomical labeling of pulmonary tree structures via deep point-graph representation-based implicit fields. *Medical Image Analysis*, 99:103367.

- Yeh, H. C. and Schum, G. M. (1980). Models of human lung airways and their application to inhaled particle deposition. *Bulletin of Mathematical Biology*, 42(3):461–480.
- Zhang, M., Li, C., Xie, F., Liu, Y., Zhang, H., Wu, J., Zhang, C., Yang, J., Sun, J., Yang, G.-Z., and Gu, Y. (2025). AirMorph: Topology-preserving deep learning for pulmonary airway analysis. *arXiv preprint arXiv:2412.11039*.
- Zhang, M., Wu, Y., Zhang, H., Qin, Y., Zheng, H., et al. (2023). Multi-site, multi-domain airway tree modeling. *Medical Image Analysis*, 90:102957.
- Zhang, M., Zhang, H., You, X., Yang, G.-Z., and Gu, Y. (2024). Implicit representation embraces challenging attributes of pulmonary airway tree structures. In *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, volume 15001 of *Lecture Notes in Computer Science*, pages 546–556. Springer.
- Zhao, H., Jiang, L., Jia, J., Torr, P. H. S., and Koltun, V. (2021). Point transformer. In *IEEE/CVF International Conference on Computer Vision (ICCV)*.
- Zhao, X., Ju, Y., Liu, C., Li, J., Huang, M., Sun, J., and Wang, T. (2009). Bronchial anatomy of left lung: a study of multi-detector row CT. *Surgical and Radiologic Anatomy*, 31(2):85–91.